A Notebook on Control System

Lu Sun, and many more.

October 30, 2026

To my family, friends and communities members who have been dedicating to the presentation of this notebook, and to all students, researchers and faculty members who might find this notebook helpful.

Contents

I. Control System Basics **1**

1. Classic Control System **3**

2. Multivariable Control System **5**
2.1. Brief Review of SISO System 5
2.1.1. SISO Plant 5
2.1.2. SISO Closed-Loop System 6
2.1.3. Stability 7
2.2. Multivariable System 7
2.2.1. Transfer Function Matrix 7
2.2.2. Calibration 8

3. Modern Control System **9**

II. Computer Controlled System **11**

4. Digital Signal Processing **13**
4.1. Digital Versus Analog Systems 13
4.2. Signal Basics 14
4.2.1. Channel and Dimension 14
4.2.2. Continuous, Discrete and Digital Signals 14
4.2.3. Deterministic and Random Signals 15
4.2.4. Symmetric and Antisymmetric Signals 15
4.2.5. Periodic and Non-periodic Signals 16
4.2.6. Energy and Power 16
4.2.7. Commonly Used Elementary Signals 16
4.2.8. Simple Signal Transformation 18
4.3. Digital System 18
4.3.1. Input-Output Model 19
4.3.2. Different Types of Digital Systems 19
4.4. Frequency Domain Analysis 19
4.5. Sampling and Reconstruction of a Signal 19
4.6. DFT 19
4.7. Digital Filtering 19

5. Computer Controlled System **21**

III. Advanced Control Technology **23**

6. Optimal Control System **25**
6.1. Static Optimization . . . 25
6.1.1. Optimization without Constraint . . . 25
6.1.2. Optimization with Equality Constraint . . . 26
6.1.3. Optimization with Equality and Inequality Constraints . . . 30
6.2. Discrete Optimal Control Basics . . . 31
6.2.1. Problem Formulation . . . 31
6.2.2. General Solution . . . 33
6.2.3. Discrete Linear Quadratic Regulator . . . 33
6.2.4. Suboptimal Discrete Linear Quadratic Regulator . . . 34
6.3. Continuous Optimal Control Basics . . . 35
6.3.1. Problem Formulation . . . 35
6.3.2. General Solution . . . 35
6.3.3. Continuous Linear Quadratic Regulator . . . 36
6.3.4. Suboptimal Continuous Linear Quadratic Regulator . . . 36
6.4. Linear Quadratic Regulator Extensions . . . 37
6.4.1. Safety Margin . . . 37
6.4.2. Tracking Problem . . . 37
6.5. Linear Quadratic Gaussian . . . 37
6.6. H_2 and H_∞ Control . . . 38
6.7. Numerical Optimal Control . . . 38

7. Robust Control System **39**

8. Model Predictive Control System **41**
8.1. Introduction . . . 41
8.1.1. Model and Constraint . . . 41
8.1.2. Problem Formulation . . . 42
8.1.3. Control Objectives . . . 46
8.2. State-Space Model Predictive Control . . . 46
8.3. Input-Output Model Predictive Control . . . 46
8.4. Robust Model Predictive Control . . . 46
8.5. Distributed Model Predictive Control . . . 46

9. Adaptive Control System **47**
9.1. Introduction . . . 47
9.1.1. A Brief History of Adaptive Control System . . . 47
9.1.2. Conventional Control System . . . 48
9.1.3. Adaptive Control Schema . . . 49
9.1.4. A Typical Adaptive Control Problem Formulation . . . 50
9.2. Parameter Estimation . . . 53
9.2.1. Least Squares Estimation . . . 53
9.2.2. Statistics Properties of LS Estimation . . . 56

9.2.3. Recursive LS Estimation . . . 57
9.2.4. LS Estimation with Time-Varying Parameters . . . 59
9.2.5. Simplified LS Estimation Methods . . . 60
9.2.6. Plant Models for LS Estimation . . . 62
9.2.7. Practical Issues in Parameter Estimation . . . 65

IV. System Identification **67**

10. Parameter and State Estimation **69**

11. Optimal State Estimation **71**
11.1. Weighted Least Squares Estimation . . . 71
11.2. Kalman Filter . . . 73
11.3. Unscented Kalman Filter . . . 76
11.3.1. Unscented Transform . . . 77
11.3.2. Unscented Kalman Filter . . . 82

12. Robust State Estimation **87**

13. Adaptive State Estimation **89**

V. Expanded Research Areas **91**

14. Fuzzy Control System **93**
14.1. Introduction . . . 93
14.2. Fuzzy Set and Fuzzy Relation . . . 93
14.2.1. Fuzzy Set . . . 93
14.2.2. Fuzzy Set Operations . . . 95
14.2.3. Commonly Used Membership Functions . . . 96
14.2.4. Fuzzy Relation . . . 97
14.3. Fuzzy Control System . . . 97
14.3.1. Fuzzification . . . 97
14.3.2. Fuzzy Interface . . . 97
14.3.3. Fuzzy Controller Design . . . 97
14.3.4. Fuzzy Controller Performance Analysis . . . 98
14.4. Fuzzy Modeling and Fuzzy System Identification . . . 98
14.5. Fuzzy System Auto-tuning . . . 98
14.6. Fuzzy Control System Design Using MATLAB . . . 99
14.6.1. Example: Vibration Detection . . . 99

15. Multi-Agent Control System **109**

A. Matrix Algebra **111**
A.1. Basic Operators . . . 111

A.2. Partitioned Matrix 112
A.3. Quadratic Forms 113
A.4. Matrix Calculus 113

B. Brief Introduction to Calculus of Variations **117**
B.1. Problem Formulation 117
B.2. Brief Discussion of the Solution 117

Foreword

I was not a big supporter for sharing notebooks. Notebooks, in my view, had been private and practically useless to anyone other than their owner.

It traces back to the summer of 2009 when I started my third year as a high school student. In the end of August when the results of Gaokao (National College Entrance Examination of China, annually held in July) came out, photocopy shops would start selling notebooks photocopies that they claimed to be from the top scorers of the exam. Much curious as I was about what these notebooks look like, never have I purchased any of them, and nor expected myself to actually learn anything from them. In fact, many of these notebooks were more difficult to read than the textbooks. I guess we could not blame the top scorers for being so smart that they sometimes made things extremely brief or overwhelmingly comprehensive. Not to mention, why would I want to adapt to notebooks of others when I had my own notebooks which in my opinion should be just as good as theirs.

However, my mind changed after becoming an undergraduate student in 2010 with the school of electrical engineering. There were so many engineering modules, and as an unfortunate result it was very challenging to dig deep into all of them evenly. I naturally put less effort on "side" modules such as material science than my major's core modules such as analog electronics. The situation became worse when I started pursuing my Ph.D. in 2014. As I had to focus on specific research areas entirely, I could hardly split enough time to systematically study those less-relevant yet interesting contents. For example, my research area was on robust statistics, state estimation and adaptive control, and I could not spend as much time as I wished on graph theory. I needed a way to quickly scratch the surface of some areas, and became decently familiar with them. There was no intention to become an expert on these areas by all means. I just wanted to understand what they are, and see whether they could have inspired my own research.

To make a difference, I forced myself to read articles beyond my comfort zone and it ended up motivating me to take notes to consolidate the knowledge. I started with easy subjects. My very first notebook was on Numerical Analysis, an entrance-level module for engineering background graduate students. Revisiting that module did not bring much benefit to my research, but it served as a good starting point. Following that were some more math notebooks, then a few on computing, operating systems and databases, and then on data science, and then on ontology.

Eventually at one point, it suddenly came to me: these notebooks that I had taken while learning these subjects as an armature, they were exactly the materials I needed, quoting myself, to "quickly scratch the surface of some areas, and became decently familiar with them". Well, in that case, why not digitizing them, making them accessible online and open-source and letting everyone read and edit it?

Hence, this is where we are.

Similar with most open-source software, these notebooks do not come with any "warranty" of any kind, meaning that there is no guarantee that everything in the notebooks is correct. Always bear in mind that the notebooks are not peer reviewed. **Do NOT cite this notebook or any notebooks in this series in your academic research paper!** If you find anything helpful here and you want to use it in your research, please trace back to the origin of the knowledge and confirm by yourself.

This notebook is suitable as:

- a quick reference guide;
- a brief introduction for beginners of an area;
- a "cheat sheet" for students or teachers for an entrance-level course.

This notebook is NOT suitable as:

- a direct research reference;
- a replacement of the corresponding textbook.

Again, this notebook is NOT peer reviewed. It is meant to be easy to read, not to be comprehensive and very rigorous.

Although this notebook is open-source, the reference materials of this notebook, including textbooks, journal papers, conference proceedings, etc., may not be open-source. Very likely many of these reference materials are licensed or copyrighted. Please legitimately access these materials and properly use them, should you decided to trace the origin of the knowledge.

Some of the figures in this notebook are plotted using Excalidraw, a very convenient tool to emulate hand drawings. The Excalidraw project can be found on GitHub, *excalidraw/excalidraw*. Other figures may come from MATLAB, R, Python, and other computation engines. The source code to reproduce the results are likely to be included in the same repository of the notebook, but there might be exceptions.

This work might have benefited from the assistance of large language models, which are used exclusively for editing purposes such as correcting grammar and rephrasing sentences, without introducing new content, generating innovative information, or changing the original intent of the text.

Preface

I approach the preparation of this notebook with some ambivalence. The principal reason is that my doctoral research was conducted in control system, focusing primarily on robust state estimation and with additional work on adaptive control system. This background may give rise to at least the following two issues.

First, the distinction between a notebook that adheres strictly to its source materials and one that incorporates the author's (my) own interpretations may become blurred. Over the course of my research, I have developed personal understandings of, and in some cases broader philosophical perspectives on, many of the topics discussed in this notebook. Such views may inevitably invade the text even when this is not my conscious intention. A further concern is that many of these ideas have not been subjected to peer review, with the exception of material already published in my academic papers. Consequently, their correctness cannot be guaranteed. Indeed, some may prove inaccurate, particularly where they are overly speculative or "visionary" in nature.

Second, there is a risk that the notebook may become more extensive and detailed than its intended purpose requires. Ideally, it should remain clear, concise, and accessible. However, my tendency is to investigate a subject in increasing depth and to include more discussion and interpretation than may be strictly necessary. Although readers with a similar research background may find such material joyful to read, it may appear redundant or of limited practical value to a broader audience.

Nevertheless, notebooks in this series have already covered on a wide range of mathematical topics, and applied mathematics should also be represented. In my view, control systems constitute one of the most important applications of mathematics. It is therefore appropriate to include the subject here. The effectiveness of this approach will become apparent as the notebook develops.

There is one more thing worth mentioning. The readers may find overlaps among some of the chapters in this notebook. For example, parameter estimation and state estimation are introduced in control systems chapters, and they are introduced again with more details in their dedicated chapters in later part of the notebook. This is to make sure that each chapter is as much self-contained as possible. The readers shall feel free to skip repeated contents when reading this notebook.

A list of key references of this notebook is given below. These materials are frequently cited in the notebook. For convenience, they are not given in the Reference section but listed here instead, whereas otherwise I will have to cite them everywhere in the text.

Control System

- Åström, Karl Johan, and Richard Murray. Feedback systems: an introduction for scientists and engineers. Princeton university press, 2021.
- Åström, Karl J., and Björn Wittenmark. Computer-controlled systems: theory and design. Courier Corporation, 2013.
- Lewis, Frank L., Draguna Vrabie, and Vassilis L. Syrmos. Optimal control. John Wiley & Sons, 2012.
- Rawlings, James B., David Q. Mayne, and Moritz M. Diehl. Model predictive control: theory, computation, and design, 2020.
- Åström, Karl J., and Björn Wittenmark. Adaptive Control. 2nd ed. Mineola, NY: Dover Publications, 2013.

System Identification

- Hampel, Frank R., Elvezio M. Ronchetti, Peter J. Rousseeuw, and Werner A. Stahel. Robust Statistics: The Approach Based on Influence Functions. New York: Wiley, 1986.
- Lewis, Frank L., Lihua Xie, and Dan Popa. Optimal and robust estimation: with an introduction to stochastic control theory. CRC press, 2017.

List of Figures

2.1. A closed-loop system with feedforward and feedback controllers. 6

4.1. Block diagram of a typical digital controller system. 13
4.2. Continuous, discrete and digital signals. 15
4.3. Example of energy and power signals, where the top plot shows an energy signal, the middle a power signal, and the bottom neither. 17
4.4. A demonstration of down-sampling. 18

6.1. A demonstration of static optimization with equality constraint problem formulation. 27
6.2. Graphical explanation to (6.7). 28

9.1. Adaptive control system general schema. 48
9.2. Gain scheduling schema. 49
9.3. MARS schema. 50
9.4. STR schema. 51
9.5. Dual control schema. 51
9.6. LTI system impulse response demonstration. 63

14.1. Membership function of fuzzy sets “young adult” and “middle-aged adult”. 94
14.2. Commonly used membership functions. 96
14.3. A simple schema for a fuzzy control system. 98
14.4. Membership functions for 3 fuzzy sets “low”, “fine” and “high” defined on input signal oscillation frequency. 100
14.5. Membership functions for 2 fuzzy sets “low” and “fine” defined on input signal oscillation amplitude. 100
14.6. Membership functions for 4 fuzzy sets “very unlikely”, “unlikely”, “likely” and “very likely”, defined on input signal oscillation amplitude. 101
14.7. Measured torque. 104
14.8. Measured torque single-sided amplitude spectrum. 105
14.9. Likelihood of vibration as a function of oscillation frequency and amplitude. 106
14.10. Likelihood of vibration as a function of oscillation frequency and amplitude using `mamfis`. 107

B.1. A demonstration of the calculation of $dx(T)$ from calculus of variations perspective. 118

List of Tables

14.1. Commonly used fuzzy set operations. 96

Part I.

Control System Basics

1. Classic Control System

2. Multivariable Control System

The classical control framework introduced in Chapter 1 assumes a linear single-input-single-output (SISO) system. This chapter extends that framework to multivariable systems, i.e., systems with multiple inputs and multiple outputs. As in the SISO case, input-output models and transfer functions are used to describe system behavior. However, for multivariable systems these take the form of transfer function matrices, which serve as the basis for controller design within the classical approach.

It is important to note that the modern control framework, introduced in the next Chapter 3, assumes multi-input multi-output (MIMO) systems by default. Modern control employs state-space models rather than transfer function matrices to represent both the plant and the controller. This modeling paradigm is generally more powerful, flexible, and intuitive, and it often leads to more straightforward implementations. Moreover, many advanced control methodologies such as optimal control, robust control and adaptive control are built directly upon the state-space formulation. The state-space model is also more effective in modeling non-linear systems or systems with non-zero initial states. For most practical MIMO applications, the methods presented in Chapter 3 are preferred over those introduced here.

Nevertheless, this chapter is included to provide historical context and to illustrate how multivariable control was originally developed within the classical framework.

2.1. Brief Review of SISO System

This section gives a brief review of important results from Chapter 1 which were derived from SISO systems. Only conclusions are given, and the proofs are neglected.

2.1.1. SISO Plant

Consider continuous time domain input-output model of a linear SISO plant. A demonstrative example is given below.

$$\ddot{y} + a_1\dot{y} + a_2 y \quad = \quad b_1\dot{u} + b_2 u \tag{2.1}$$

where y and a are the output and the input respectively, and the dot over a variable refers to the differential of that variable over time. The transfer function corresponding with (2.1) can be easily derived by applying Laplace transform on both side of the

equation to get

$$\begin{aligned} G(s) &= \frac{Y(s)}{U(s)} \\ &= \frac{b_1 s + b_2}{s^2 + a_1 s + a_2} \end{aligned} \tag{2.2}$$

In this example, both the nominator and the denominator of the transfer function (2.2) are polynomials, and the degree of the denominator w.r.t. s is higher than that of the nominator. Hence, $G(s)$ is rational and strictly proper. The roots of the nominator and the denominator are known as the zeros and poles of the transfer function.

2.1.2. SISO Closed-Loop System

A SISO closed loop system may look like Fig. 2.1. Notice that the feedforward controller given in blue is optional. The feedforward controller is mainly used to mitigate the effect of known disturbance more effectively than the feedback controller, the later of which can also perform the task but would often introduce more latency. The disturbance can be constant or dynamic, and may affect the plant in several different ways, such as affecting its input, its output, or its intermediate values.

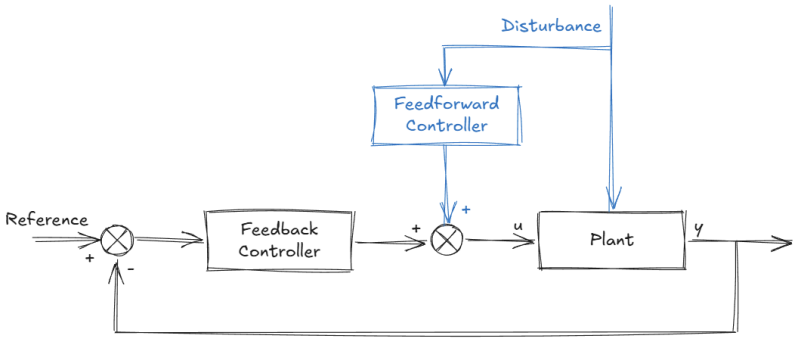


Figure 2.1.: A closed-loop system with feedforward and feedback controllers.

In the following discussion, assume zero disturbance and ignore the feedforward controller. Let the transfer function of the feedback controller be given by $K(s)$. The open-loop transfer function is therefore given by

$$L(s) = G(s)K(s)$$

whereas the closed-loop transfer function is given by

$$H(s) = \frac{Y(s)}{R(s)} = \frac{G(s)K(s)}{1 + G(s)K(s)} = \frac{L(s)}{1 + L(s)} \tag{2.3}$$

where $R(s)$ is the Laplace transform of the reference signal.

2.1.3. Stability

Given a transfer function, whether it describes the plant (2.2) or the closed-loop system (2.3), its corresponding system is bounded-input-bounded-output (BIBO) stable if its poles have strictly negative real parts, and its output is bounded as long as its input is bounded in magnitude.

However, for a BIBO stable closed-loop system, it may not be internally stable. It is possible that for a bounded reference signal $r(t)$, though the output $y(t)$ is bounded, the control signal $u(t)$ is unbounded. Such system will eventually crash in practice, the actuator will fail to supply infinitely large input to the plant. That could happen if there is unstable pole-zero cancellation between $G(s)$ and $K(s)$, and $K(s)$ has unstable poles but canceled out by the zeros of $G(s)$. In the remainder of this chapter, for the simplicity of the discussion, it is assumed that there is no unstable pole-zero cancellation, and the system is always internally stable as long as it is BIBO stable.

The poles of the closed-loop system is given by the roots of $1 + L(s)$. Therefore, the stability of the system can be revealed by the open-loop transfer function. See Nyquist theorem in Chapter 1 for the details. The phase margin and gain margin are often used to describe how "far" a stable system is to instability, and they are calculated by

$$\begin{aligned} \psi &= \arg\left(L(j\omega_c)\right) + \pi \\ \text{s.t.} \quad & |L(j\omega_c)| = 1 \end{aligned}$$

and

$$\begin{aligned} \mu &= \frac{1}{|L(j\omega_d)|} \\ \text{s.t.} \quad & \arg\left(L(j\omega_d)\right) = \pi \end{aligned}$$

respectively, where $\arg(\cdot)$ returns the principal angle value $(-\pi, \pi]$ of a complex number. The larger the margins, the "safer" the system.

2.2. Multivariable System

A **multivariable system** refers to a **multiple-input-multiple-output** (MIMO) system. Let m and l denote the input and output numbers respectively. The system is called a **square system** if $m = l$.

2.2.1. Transfer Function Matrix

The input-output model of a multivariable system can be described by the transfer function matrix $G(s)$ as follows

$$Y(s) = G(s)U(s)$$

where $Y(s)$ and $U(s)$ are vectors with length l and m respectively, and $G(s)$ an $l \times m$ matrix. The elements in $G(s)$ such as $g_{ij}(s)$ is the transfer function relating the i-th output to the j-th input.

The transfer matrix $G(s)$ is said to be a rational transfer function matrix if each and every $g_{ij}(s)$ is rational, and it is (strictly) proper if each and every $g_{ij}(s)$ is (strictly) proper.

For a system with rational and proper transfer function matrix, there is such a **state-space realization** of the system

$$\begin{aligned} \dot{x} &= Ax + Bu \\ y &= Cx + Du \end{aligned}$$

that

$$G(s) = C(sI - A)^{-1}B + D$$

where I is the identity matrix. Notice that the state-space realization is not unique if there is no other restrictions.

2.2.2. Calibration

Given a linear and zero-state multivariable system, system identification is often required to calibrate $G(s)$. A practical approach is to investigate the input-output relation per input and output combination by applying unit signal to a specific input while keeping other inputs zero.

3. Modern Control System

Part II.

Computer Controlled System

4. Digital Signal Processing

Digital signal processing is the basis of computer controlled system.

This chapter servers as a brief introduction to digital signal processing, including signal sampling and reconstruction, commonly seen discrete signals and their features, z-transform, frequency domain analysis, discrete Fourier transform and digital filtering.

4.1. Digital Versus Analog Systems

It has become a common practice nowadays to use digital controllers over analog controllers as the preferable choice in most occasions. A commonly used block diagram of a digital processing system looks like Fig. 4.1. The digital and analog modules are given by black and red respectively. The analog-to-digital converter (ADC) and the digital-to-analog converter (DAC) bridges the digital and analog worlds, and they are colored in orange.

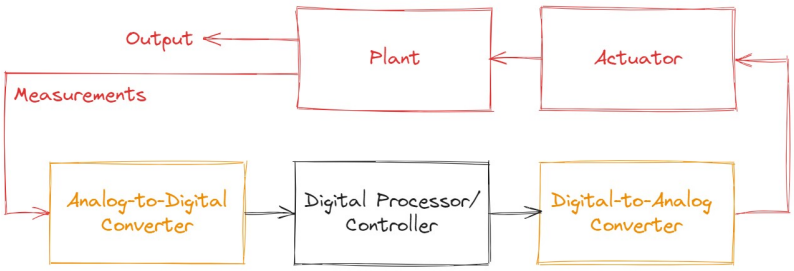


Figure 4.1.: Block diagram of a typical digital controller system.

A digital processing system has the following advantages over a conventional analog processing system.

- More flexible. Digital processor is usually re-programmable, thus making it cheaper and faster to scale than an analog system.
- Easily reproducible. With standard hardware setup, reproducing a digital system can be as simple as copy-and-paste the code and it is much easier than re-building an analog system.
- Consistent performance. The precision, accuracy, responding time, etc., of a digital system is more deterministic and consistent than an analog system.

However, digital system is not always an superior alternative to the analog system. Comparing with an analog system, a digital system may have the following drawbacks.

Due to the use of ADC, DAC, digital signature processor (DSP), etc., a digital system can introduce a "capital cost". For very simple and mature applications, using digital system could be more expensive. ADC, DAC and DSP also introduce delay. Though these units are becoming faster and faster, there are still tasks where the speed and the bandwidth of a digital system is too slow. In such cases, analog system or optical system might be a better choice.

4.2. Signal Basics

Basic concepts as well as commonly seen elementary signals are introduced.

4.2.1. Channel and Dimension

Signals, both analog and digital, can be scalar or vector. A signal is often formulated as a function of time, location, etc. The number components in the signal and the number of independent variables formulating the signal are knows as the **signal channel** and **signal dimension** respectively. A signal with vector size n and number of independent variables m is known as a n-channel-m-dimensional signal. An example is given below.

Consider a colored image

$$I(x,y) \quad = \quad \begin{bmatrix} I_r(x,y) \\ I_g(x,y) \\ I_b(x,y) \end{bmatrix}$$

where I_r, I_g, I_b and (x,y) are the RGBs and the coordinate of the pixels respectively. This is a 3-channel-2-dimensional signal.

Though multi-channel-multi-dimensional signals are very common, for the convenience of study, this chapter will mostly focus on 1-channel-1-dimensional signals.

4.2.2. Continuous, Discrete and Digital Signals

Consider a signal formulated as a function of time. If the signal is continuous in both the time scale and the value scale, it is known as a **continuous signal**. Most signals in our daily life are continuous by nature. For example, the speed of a car can be formulated as a continuous signal.

Consider sampling a continuous signal by recording its values in uniformly distributed timestamps. The time scale of the signal becomes discrete and countable. A signal of such type is known as a **discrete-time signal**, or sometimes **discrete signal** for short.

Consider mapping the value of a continuous signal to a set of probable values, usually values with finite digits. The value scale of the signal becomes discrete. A signal of such type is known as a **discrete-valued signal**.

A signal that is both discrete-time and discrete-valued, i.e., has both discrete time and value scales, is also known as a **digital signal**.

A figure to illustrate the relation of the above signals is given in Fig. 4.2.

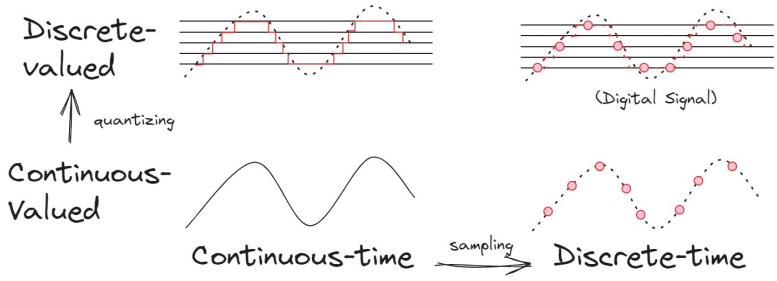


Figure 4.2.: Continuous, discrete and digital signals.

In this notebook, sometimes discrete signal and digital signal are used interchangeably, assuming that error introduced by value mapping is neglectable.

4.2.3. Deterministic and Random Signals

There is always random noise and stochastic disturbances in a physical system. From this point of view, no system or signal is deterministic. In practice, as long as the signal is precise and accurate enough to describe the system status, it is considered a **deterministic signal**. Otherwise if the signal is unknown or contains considerably large noise, it is considered a **random signal**.

Deterministic signals and models are relatively easier to analyze. When it comes to random signals and stochastic systems, probability and statistics based analysis are usually adopted.

4.2.4. Symmetric and Antisymmetric Signals

A signal $x(n)$ with n the time index is a **symmetric signal** (also known as **even signal**) if

$$x(n) \quad = \quad x(-n)$$

A signal $x(n)$ is an **asymmetric signal** (also known as **odd signal**) if

$$x(n) \quad = \quad -x(-n)$$

Given any signal $x(n)$, define

$$\begin{aligned} x_e(n) &= \frac{1}{2}\left[x(n) + x(-n)\right] \\ x_o(n) &= \frac{1}{2}\left[x(n) - x(-n)\right] \end{aligned}$$

It can be easily verified that $x_e(n)$ and $x_o(n)$ are even and odd signals respectively, and $x(n) = x_e(n) + x_o(n)$. Therefore, we can conclude that any signal can be decomposed into the symmetric and the asymmetric portions.

4.2.5. Periodic and Non-periodic Signals

A discrete signal $x(n)$ with n the time index is a **periodic signal** if there is such N that

$$x(n) \quad = \quad x(n+N), \text{ for all } n$$

and the smallest $N > 0$ is the **period** of the signal. Notice that a constant signal $x(n) = c$ is often taken as a periodic signal as well, but of non-definable period. In the remaining of our discussion, we shall consider the "regular" periodic signal with $N > 0$ unless otherwise mentioned.

A signal without such N is known as a **non-periodic signal**. A non-periodic signal may have one or more periodic components, and can be taken as a sum of the periodic and non-periodic components. In some studies, non-periodic signals are taken as a limiting case of periodic signal with infinite period.

4.2.6. Energy and Power

Let $x(n)$ be a discrete signal with n the time index. The **energy of a signal** and the **power of a signal** are defined by

$$\begin{aligned} E &= \sum_{n=-\infty}^{\infty} |x(n)|^2 \\ P &= \lim_{N\to\infty} \frac{1}{2N+1} \sum_{n=-N}^{N} |x(n)|^2 \end{aligned}$$

respectively. A signal with finite energy is known as an **energy signal**. A signal with finite power is known as a **power signal**. From the definition, we know that an energy signal must have a power of zero. Examples are given in Fig. 4.3.

It can be proved easily that a period signal has finite power as long as $x(n)$ is bounded. The energy, on the other hand, is infinite unless $x(n) = 0$ is a zero signal.

4.2.7. Commonly Used Elementary Signals

Let $x(n)$ be a discrete signal with n the time index. Commonly used elementary signals are introduced below.

The unit sample sequence is given by

$$\delta(n) \quad = \quad \begin{cases} 1 & n = 0 \\ 0 & \text{otherwise} \end{cases}$$

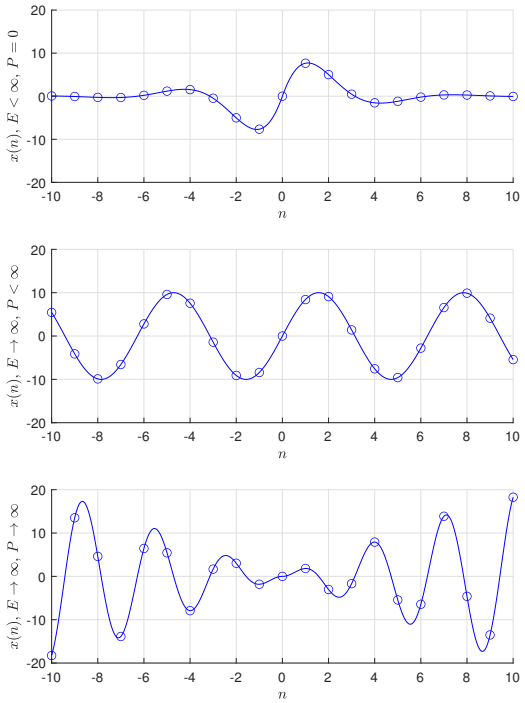


Figure 4.3.: Example of energy and power signals, where the top plot shows an energy signal, the middle a power signal, and the bottom neither.

The unit step signal is given by

$$u(n) = \begin{cases} 1 & n \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

The unit ramp signal is given by

$$u_r(n) = \begin{cases} n & n \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

The exponential signal is given by

$$x(n) = a^n$$

where notice that a can be a complex value denoted by $a = re^{j\theta}$, in which case

$$x(n) = r^n e^{j\theta n} = r^n \left(\cos(\theta n) + j\sin(\theta n)\right)$$

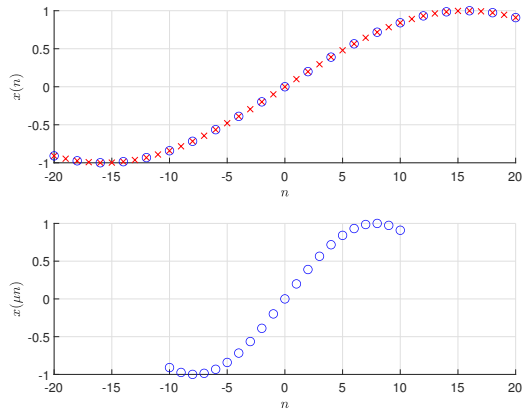


Figure 4.4.: A demonstration of down-sampling.

4.2.8. Simple Signal Transformation

For a general discrete signal $x(n)$, $x(-n)$ is a fold of the signal at $n = 0$. We can easily prove that $x(-n + 2k)$ is a fold of the signal at $n = k$.

Signal $x(n - k)$ is a shift of the original signal $x(n)$ to the right by k samples, i.e., it is $x(n)$ delayed by k samples. Likewise, $x(n + k)$ is the original signal to shift to the left by k samples.

Signal $x(\mu n)$, $\mu \in \mathbb{N}^+$ is a time-scaling or down-sampling of the original signal $x(n)$. Graphical wise, it "squeezes" the signal horizontally, and some information may be lost from the squeezing. A demonstrative figure is given in Fig. 4.4. The values captured by the blue circle is the down sampled signal sequence. In this example, $\mu = 2$, i.e., one out of two samples are captured. The information contained in the original signal between two new samples, i.e., the samples in red cross without blue circle, is lost.

4.3. Digital System

A **digital system** maps discrete signal $x(n)$ to another discrete signal $y(n)$. It can be described by

$$y(n) \quad = \quad \mathcal{T}\left[x(n), x(n-1), \ldots\right]$$

where system is denoted by $\mathcal{T}$.

Notice that in this notebook, unless otherwise mentioned, only **casual system** is considered where the system output depends on the current and the past inputs, but not future inputs. In this case, $y(n)$ may depend on the current and the past inputs.

4.3.1. Input-Output Model

There are varieties of ways to model the system. A commonly used way is to use an input-output model described by the difference equation. A simple example is given below.

$$y(k) = \sum_{i=-\infty}^{k-1} x(i) = y(k-1) + x(k-1) \tag{4.1}$$

Equation (4.1) calculates the aggregated sum of the input signal. For this system, we can see that its output $y(k)$ can be entirely described by its previous value $y(k-1)$ and the current input $x(k)$. This is because all the "historical information" priori to $k-1$ that is useful in calculating $y(k)$ is contained in $y(k-1)$.

In general, if the current status of a system can be formulated as a function of the one-step previous status of the system and the current input and noise, the system model is known as a *Markov process*. Markov process is widely used in modeling the system behavior. The famous state-space realization of a system is a systematic way of using a Markov process to model a system.

By tracing back in time, we can find the "initial state" of a Markov process. In the context of (4.1), initial state can be denoted as $y(0)$. We do not care the values of $y(k)$ and $x(k)$ priori to $k = 0$. From the recursive equation (4.1), it is obvious that $y(k)$ is essentially determined by the initial state $y(0)$ and the historical inputs $x(0), ..., x(k-1)$.

For the convenience of analysis, it is quite common that zero initial state (also known as relaxed initial state) $y(0) = 0$ is assumed, which allows us to focus on the study of input-output relationship. The system response, in which case only affected by the inputs $x(0), ..., x(k-1)$, is known as the *zero state response*. Likewise, there is the *zero input response* of the system where non-zero initial state $y(0) \neq 0$ and zero input $x(k) = 0$ are considered.

4.3.2. Different Types of Digital Systems

4.4. Frequency Domain Analysis

4.5. Sampling and Reconstruction of a Signal

4.6. DFT

4.7. Digital Filtering

5. Computer Controlled System

Part III.

Advanced Control Technology

6. Optimal Control System

In a typical optimal control problem formulation, a cost function is defined. The cost function is often made positively correlated with the regulation error and the control signal energy. Such control that minimizes the cost function is calculated. The key to the optimal control is to formulate the problem appropriately and to find the correct mathematical tools to solve the problem.

Optimization problem formulation and optimal control are introduced in this chapter.

6.1. Static Optimization

Assume static model where the cost function $L(u)$ (or L for simplicity) is a function of u and does not change over time. There may or may not be additional constraints on u. If there are constraints, the constraints do not change over time either. The objective is to find the signal u that minimizes L.

Notice that $u \in \mathbb{R}^m$ is a vector containing m independent variables, and $L \in \mathbb{R}$ is a scalar. It is assumed that L is smooth, and it has derivatives of all orders on u.

6.1.1. Optimization without Constraint

Consider minimizing L without additional constraints on u. Using Taylor series expansion, the increment of L can be formulated as follows.

$$\Delta L = L_u^T \Delta u + \frac{1}{2} \Delta u^T L_{uu} \Delta u + \dots$$

where

$$\begin{aligned} L_u &= \frac{dL}{du} \in \mathbb{R}^m \\ L_{uu} &= \frac{d^2 L}{du^2} \in \mathbb{R}^{m \times m} \end{aligned} \tag{6.1}$$

about a particular choice of u. A local minimum of L is achieved under the following necessary condition

$$L_u = 0$$

in which case u is called a **critical point**, also known as **stationary point**. Furthermore, for u to be a minimum, ΔL should stay positive in the neighbor of u. A necessary condition for that could be that L_{uu} is positive definite, i.e.,

$$L_{uu} > 0$$

Consider the following example.

A Quadratic Surface Example
Let

$$L = \frac{1}{2}u^T Q u + S u \tag{6.2}$$

$$= \frac{1}{2}u^T \begin{bmatrix} q_{11} & q_{12} \\ q_{21} & q_{22} \end{bmatrix} u + \begin{bmatrix} s_1 & s_2 \end{bmatrix} u \tag{6.3}$$

Find u at the critical point, and discuss whether it is a maximum, minimum or other.

From (6.2) and (6.3),

$$\begin{aligned} L_u &= Qu + S \\ L_{uu} &= Q \end{aligned}$$

Let $L_u = 0$ to get the critical point u^* as follows.

$$u^* = -Q^{-1}S$$

Whether this critical point is a maximum, minimum, or other cases depends on L_{uu}.

- If $L_{uu} > 0$, i.e. Q is positive definite, the critical point is a minimum.
- If $L_{uu} < 0$, i.e. Q is negative definite, the critical point is a maximum.
- Otherwise,
  - If $|L_{uu}| < 0$, i.e. $q_{11}q_{22} - q_{12}q_{21} < 0$, the critical point is a saddle point.
  - If $|L_{uu}| = 0$, the critical point is a singular point and more information is required to determine the shape at the critical point.

6.1.2. Optimization with Equality Constraint

Consider the optimization with equality constraint as follows.

$$\text{minimize} \quad L(u) \tag{6.4a}$$

$$\text{s.t.} \quad f(u) = 0 \tag{6.4b}$$

where $u \in \mathbb{R}^m$ and $f(u) \in \mathbb{R}^n$ (or f for simplicity), and without losing generality we assume that $n < m$ (otherwise u might be solvable from $f = 0$).

It is obvious that the solution to (6.4a) would not be necessarily at $L_u = 0$. As a matter of fact, at any point of u satisfying $f = 0$, it is not necessary that $L_u = 0$. This is demonstrated by Fig. 6.1, where L is given by the contour (assuming $u \in \mathbb{R}^2$ in this demonstration plot), and $f = 0$ the red solid line.

A necessary condition of the critical point of (6.4a) subject to (6.4b) can be interpreted as follows.

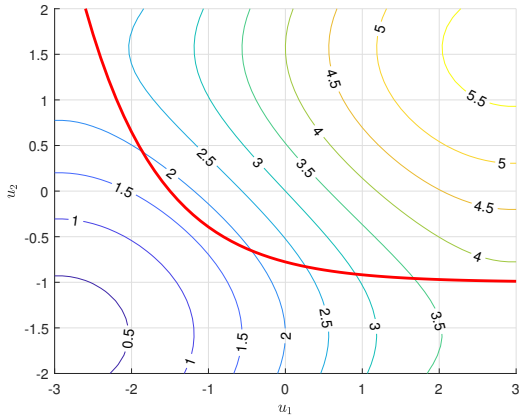


Figure 6.1.: A demonstration of static optimization with equality constraint problem formulation.

Let du be a differential about u where $f = 0$. Let L_u be given by (6.1) and $f_u = \frac{df}{du}$ the Jacobin matrix. While $dL = L_u^T du \neq 0$ in general, when du moves along $f = 0$, i.e., when $df = f_u d_u = 0$, $dL = L_u^T du = 0$ should hold true. This is expressed by

$$L_u^T du = 0 \tag{6.5a}$$

$$s.t. \quad f_u du = 0 \tag{6.5b}$$

where notice the following

- The dimensions are $L_u \in \mathbb{R}^{m\times 1}$, $L_u^T \in \mathbb{R}^{1\times m}$, $du \in \mathbb{R}^{m\times 1}$, $f_u \in \mathbb{R}^{n\times m}$.
- It is u that matters. The choice of u determines L_u and f_u. The differential du is used to bridge (6.5a) and (6.5b). It is treated as a vector space. Solving or assigning specific values to du are pointless.
- Equation (6.5b) contains n equations, and (6.5a) holds true only when all n equations are satisfied.

A necessary condition for (6.5) is given as follows. There must be such $\lambda \in \mathbb{R}^n$ with n degrees of freedom that

$$L_u^T du + \lambda^T f_u du \quad = \quad 0 \tag{6.6}$$

The explanation is given below. Equation (6.5) essentially states that if $f_u du = 0$, $L_u^T du = 0$. This is the same in (6.6), where $L_u^T du = 0$ can be derived from $f_u du = 0$, regardless of the values of λ. If $f_u du \neq 0$ even for one of the n equality constraints, (6.5) applies no restriction on the value of $L_u^T du$. This is also the same in (6.6), where for any non-zero $f_u du$, $L_u^T du$ can be any value, with λ being the coefficient.

Notice that du is a differential of u in the m-th order vector space and can be dropped from (6.6), in which case it becomes

$$L_u^T + \lambda^T f_u \quad = \quad 0 \tag{6.7}$$

In (6.7), $\lambda \in \mathbb{R}^n$ is known as the **Lagrange multipliers**. Equation (6.7) contains m equations, and it is a portion of Karush–Kuhn–Tucker (KKT) conditions which will be introduced formally in Section 6.1.3.

Equation (6.7) can also be interpreted graphically. At the critical point denoted by the red dot, moving along $f = 0$ should not increase or decrease L. This suggests that $f = 0$ shall tangent the contour at the critical point as shown in Fig. 6.2. Think of

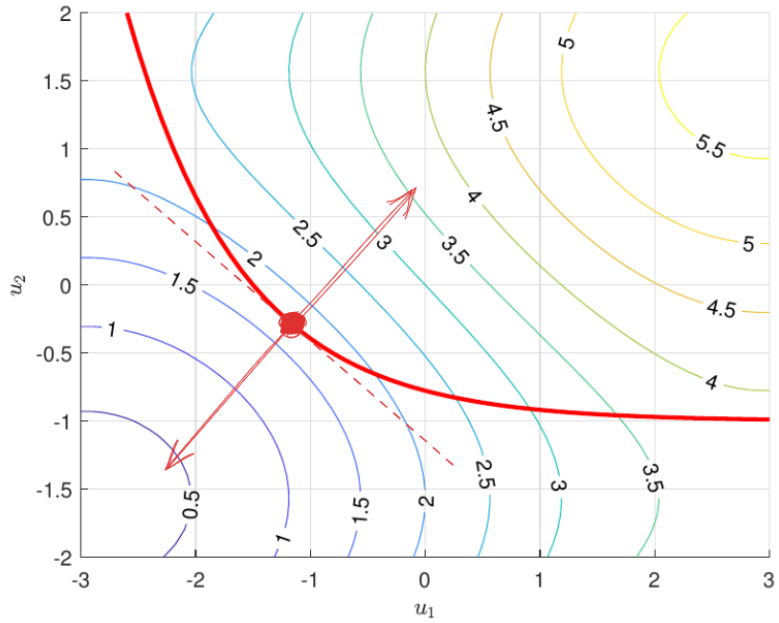


Figure 6.2.: Graphical explanation to (6.7).

$f_u = \nabla f$ and $L_u = \nabla L$ as the gradient of f and L respectively. Since the plots are tangent, the gradient must be parallel, which would suggest something like (6.7).

The necessary condition (6.4b) and (6.7) can also be formulated using **Lagrangian function** which is given below. Notice that Lagrangian function in some context may also be called **Hamiltonian function**. They are very similar mathematically, and mostly differ in the context of usage.

The Lagrangian function corresponding with problem (6.4) is given by

$$\mathcal{L}(u, \lambda) \quad = \quad L(u) + \lambda^T f(u) \tag{6.8}$$

and (6.4b), (6.7) can be represented by

$$\frac{\partial \mathcal{L}}{\partial \lambda} = 0 \tag{6.9}$$

$$\frac{\partial \mathcal{L}}{\partial u} = 0 \tag{6.10}$$

respectively.

Lagrangian function is a very useful tool in solving optimization problem with different types of constraints. It is difficult to solve the original optimization problem (6.4a) with equality constraint (6.4b). By introducing Lagrangian function (6.8) with λ and u the independent variable, the original problem is converted into an optimization problem without equality constraint, whose solution is given by (6.9) and (6.10) which are much easier to solve. The converted problem is known as the **dual problem** of the original problem.

Notice that Lagrangian function is only a necessary condition to the original question. The critical point of the Lagrangian function must be further verified. To verify that the critical point is indeed a local minimum, check the second-order derivative of $L(u)$ about the critical point u.

Consider the following example.

A Quadratic Surface with Linear Equality Constraint Example

Let

$$\begin{aligned} & L(x,u) = \frac{1}{2}x^T Q x + \frac{1}{2}u^T R u \\ \text{s.t.} \quad & f(x,u) = x + Bu + c = 0 \end{aligned} \tag{6.11}$$

where the dimensions are $x \in \mathbb{R}^n$, $u \in \mathbb{R}^m$, $Q \in \mathbb{R}^{n\times n}$, $R \in \mathbb{R}^{m\times m}$, $B \in \mathbb{R}^{n\times m}$, $c \in \mathbb{R}^n$. Find u at the critical point, and discuss whether it is a minimum.

The Lagrangian function of the problem is

$$\mathcal{L} = \frac{1}{2}x^T Q x + \frac{1}{2}u^T R u + \lambda^T (x + Bu + c)$$

where $\lambda \in \mathbb{R}^n$. The necessary condition for a critical point is

$$\frac{\partial \mathcal{L}}{\partial \lambda} = x + Bu + c = 0 \tag{6.12}$$

$$\frac{\partial \mathcal{L}}{\partial x} = Qx + \lambda = 0 \tag{6.13}$$

$$\frac{\partial \mathcal{L}}{\partial u} = Ru + B^T \lambda = 0$$

The solution is

$$u = -\left(R + B^T Q B\right)^{-1} B^T Q c \tag{6.14}$$

$$x = -\left(I - B\left(R + B^T Q B\right)^{-1} B^T Q\right) c \tag{6.15}$$

$$\lambda = \left(Q^{-1} + B R^{-1} B^T\right)^{-1} c$$

The verify whether it is a minimum w.r.t. u, check the following

$$L_{uu} = R + B^T Q B$$

which is positive definite as long as Q and R are positive definite. Indeed, this is a minimum w.r.t. u. The minimum value can be obtained by substituting (6.14) and (6.15) into (6.11).

$$\begin{aligned} L^* &= \frac{1}{2} c^T \left(Q^{-1} + B R^{-1} B^T \right)^{-1} c \\ &= \frac{1}{2} c^T \lambda \end{aligned}$$

6.1.3. Optimization with Equality and Inequality Constraints

Consider the optimization problem with both equality and inequality constraints as follows.

$$\begin{aligned} \text{minimize} \quad & L(u) \\ \text{s.t.} \quad & f(u) = 0 \\ & g(n) \leq 0 \end{aligned} \tag{6.16}$$

where $u \in \mathbb{R}^m$, $f(u) \in \mathbb{R}^p$, $g(u) \in \mathbb{R}^q$.

Obviously, the solution of the problem may reside at either $g(u) = 0$ or $g(u) < 0$. Therefore, an intuitive way of solving the problem is to use "trail-and-error" as follows. Ignore the inequality constraint (6.16), and the problem becomes an optimization with equality constraint problem. Test the obtained solution using (6.16). If (6.16) is satisfied, the solution to the new problem is also a solution to the original problem. Otherwise, reformulate the problem and let both $f(u) = 0$, $g(u) = 0$ be equality constraints.

Inspired by this idea, the optimization problem with both equality and inequality constraints can be solved as follows. First, define generalized Lagrangian function

$$\mathcal{L}(u, \alpha, \beta) = L(u) + \alpha^T f(u) + \beta^T g(u)$$

where $\alpha \in \mathbb{R}^p$ and $\beta \in \mathbb{R}^q$ are Lagrange multipliers associated with equality and inequality constraints, respectively. In some literatures, $\lambda = \begin{bmatrix} \alpha^T & \beta^T \end{bmatrix}^T$ might be used in the notation.

The necessary condition for a solution can be obtained using **Karush-Kuhn-Tucker** (KKT) conditions as follows. The KKT conditions are essentially first derivative tests for the solution to be optimal. Notice that sometimes KKT can be necessary and sufficient condition for the solution if L, f and g meet certain criteria. This is not

discussed here in details.

$$\frac{\partial \mathcal{L}}{\partial u} = 0 \tag{6.17}$$

$$\frac{\partial \mathcal{L}}{\partial \alpha} = 0 \tag{6.18}$$

$$\beta_i g_i(u) = 0 \tag{6.19}$$

$$g_i(u) \leq 0 \tag{6.20}$$

$$\beta_i \geq 0 \tag{6.21}$$

Equations (6.17) and (6.18) are straight forward and they are the same with the previous discussed optimization with equality constraint problem. Equations (6.19), (6.20) and (6.21) provide additional β degrees of freedom to the problem formulation to express the inequality constraints. They can be interpreted as follows.

- From (6.19), anything relevant to inequality constraint never really contribute to the Lagrangian function value. Therefore, solving (6.17) and (6.18) still gives the necessary condition meeting the equality constraint.
- Equation (6.19) implies that one of β_i and $g_i(u)$ must be zero. Equations (6.19), (6.20) and (6.21) together interpret the inequality constraint as follows.
  - If the critical point lays on the ith inequality constraint boundary, i.e. $g_i(u) = 0$, the value of β_i does not matter. Without loosing generality, let $\beta_i > 0$.
  - If the critical point does not lay on the ith inequality constraint boundary, i.e. $g_i(u) < 0$, $\beta_i = 0$ is used so that (6.19) is zero.
  - In other words, if $g_i(u) < 0$, $\beta_i = 0$; if $g_i(u) = 0$, $\beta_i > 0$.

Equations (6.17) to (6.21) together form the dual problem of the original problem. Equation (6.19) is known as the complementary slackness, and (6.21) dual feasibility.

6.2. Discrete Optimal Control Basics

Optimal control is strongly connected with the static optimization problem formulation introduced in Section 6.1. In a broad sense, optimal control can be taken as a use case of the static problem formulation, with the system model treated as a constraint.

This section introduces the basics of optimal control on a discrete time system. The next Section 6.3 introduces that of a continuous time system.

6.2.1. Problem Formulation

A general optimal control of discrete system problem can be described as follows. Let the system dynamics be described by

$$x(k+1) = f^k\left(x(k), u(k)\right) \tag{6.22}$$

where $k = 0, \ldots, N$ is the time index, N the end of the computation cycle, $x(k) \in \mathbb{R}^n$ the state vector ($x(0)$ is the initial state), $u(k) \in \mathbb{R}^m$ the input and f^k the system dynamics. No disturbance or process error is considered. It is assumed that $x(k)$ is always known.

The objective of the optimization is to find such $u(k)$ that minimizes the following cost function

$$J = \phi(N, x(N)) + \sum_{k=0}^{N-1} L^k(x(k), u(k)) \tag{6.23}$$

where N is the time to reach a particular goal, $x(N)$ the final state vector at $k = N$, and L^k the cost relevant to the magnitude of $x(k)$ and $u(k)$ at time instant k. In some cases, N is a parameter to optimize, for example in a "do-it-fast" type of problem. In other cases, N is a fixed value, for example in a "do-the-best-within-given-time" type of problem.

Some commonly seen cost functions are given below.

Minimum Time

Use $u(k)$ to drive the system from its initial state to a given state. The total time consumption is minimized.

$$J = N$$

Minimum Fuel and Energy

Use $u(k)$ to drive the system from its initial state to a given state. The cost of applying the control signal is minimized. To minimize the fuel consumption,

$$J = \sum_{k=0}^{N-1} |u(k)|$$

To minimize energy,

$$J = \sum_{k=0}^{N-1} u(k)^T u(k)$$

Regulation

Use $u(k)$ to regulate the states to zero from a non-zero initial state or from state disturbances. Along the way, both regulation error and the control signal driving energy are considered.

$$J = \frac{1}{2} x(N)^T S x(N) + \frac{1}{2} \sum_{k=0}^{N-1} \left(x(k)^T Q x(k) + u(k)^T R u(k) \right)$$

where Q, S and R are weight matrices that trades of the minimization of regulation error (intermediate state error), final state error or control signal energy.

6.2.2. General Solution

A general solution to (6.23) subject to (6.22) is given below.

Define the augmented cost function as the Lagrangian function below, taking into account (6.22) as the equality constraint.

$$J' = \phi\left(N, x(N)\right) + \sum_{k=0}^{N-1} \left(L^k\left(x(k), u(k)\right) + \lambda_{k+1}^T \left(f^k\left(x(k), u(k)\right) - x(k+1)\right)\right)$$

which can be rewritten as

$$\begin{aligned} J' &= \phi\left(N, x(N)\right) - \lambda_N^T x(N) + H^0(x(0), u(0)) \\ &\quad + \sum_{k=0}^{N-1} \left(H^k\left(x(k), u(k)\right) - \lambda_k^T x(k)\right) \end{aligned}$$

where

$$H^k\left(x(k), u(k)\right) = L^k\left(x(k), u(k)\right) + \lambda_{k+1}^T f^k\left(x(k), u(k)\right)$$

is the Hamiltonian defined at each time stamp k. Apply KKT condition and the solution is

$$\begin{aligned} x(k+1) &= \frac{\partial H^k}{\partial \lambda_{k+1}} = f^k\left(x(k), u(k)\right) \\ \lambda_k &= \frac{\partial H^k}{\partial x(k)} = \left(\frac{\partial f^k}{\partial x(k)}\right)^T \lambda_{k+1} + \frac{\partial L^k}{\partial x(k)} \\ 0 &= \frac{\partial H^k}{\partial u(k)} = \left(\frac{\partial f^k}{\partial u(k)}\right)^T \lambda_{k+1} + \frac{\partial L^k}{\partial u(k)} \\ 0 &= \left(\frac{\partial L^0}{\partial x(0)} + \left(\frac{\partial f^0}{\partial x(0)}\right)^T \lambda_1\right)^T dx(0) \\ 0 &= \left(\frac{\partial \phi}{\partial x(N)} - \lambda_N\right)^T dx(N) \end{aligned}$$

6.2.3. Discrete Linear Quadratic Regulator

Linear quadratic regulator (LQR) is a special case of the general problem formulation (6.22) and (6.23). It is one of the most commonly seen optimal control problems.

Consider linear system as follows. It is assumed that the state vector is known exactly.

$$x(k+1) = A(k)x(k) + B(k)u(k)$$

where notice that the process matrix $A(k)$, input matrix $B(k)$ can be time-varying. The cost function is given by

$$J = \frac{1}{2}x(N)^T S_N x(N) + \frac{1}{2}\sum_{k=0}^{N-1}\left(x(k)^T Q_k x(k) + u(k)^T R_k u(k)\right)$$

where $|S_N| \geq 0$, $|Q_k| \geq 0$ and $|R_k| > 0$ are known symmetric matrices. In the LQR problem, the target of the control is to regulate the state vector to zero while given an non-zero initial state, and meantime tries to keep the control signal energy small.

The solution of the problem can be found by constructing the Hamiltonian function as introduced earlier in Section 6.2.2. Result is given below.

$$\begin{aligned} S_k &= A(k)^T\left(S_{k+1} - S_{k+1}B(k)\left(B(k)^T S_{k+1}B(k) + R_k\right)^{-1}B(k)^T S_{k+1}\right)A(k) \\ &\quad + Q_k \qquad (6.24) \\ K(k) &= \left(B(k)^T S_{k+1}B(k) + R_k\right)^{-1}B(k)^T S_{k+1}A(k) \qquad (6.25) \\ u(k) &= -K(k)x(k) \end{aligned}$$

where $k = 0, \ldots, N-1$. Matrix $K(k)$ in (6.25) is known as the Kalman gain (it might be more intuitive to simply call it the "control gain") and it can be calculated offline.

6.2.4. Suboptimal Discrete Linear Quadratic Regulator

Notice that $K(k)$ is time varying in general. Only if A, B, Q and R are constant and at the same time $N \to \infty$, S_k becomes a constant and so does $K(k)$. In such case, the cost function becomes

$$J = \frac{1}{2}\sum_{k=0}^{\infty}\left(x(k)^T Q x(k) + u(k)^T R u(k)\right)$$

The problem can be solved similarly with S the unique positive definite solution to the following discrete time algebraic Riccati equation

$$S = A^T\left(S - SB\left(B^T SB + R\right)^{-1}B^T S\right)A \qquad (6.26)$$

and the Kalman gain

$$K_\infty = \left(B^T SB + R\right)^{-1}B^T SA \qquad (6.27)$$

which is also known as the static Kalman gain.

In a LTI system where A, B, Q and R are constant matrices and N a finite number, dealing with time varying $K(k)$ can be problematic in practice due to the large memory and offline computation load requirement. It is sometimes preferable to rather use suboptimal control $u = -K_\infty x(k)$ instead of $u = -K(k)x(k)$, where K_∞ is calculated using (6.27), assuming $N \to \infty$ and ignoring $x(N)^T S_N x(N)$ in the cost function.

It is important to verify whether the system remains asymptotically stable when suboptimal control law is applied. Only the conclusion is given as follows: the system (A, B) is stabilizable if and only if:

- Equation (6.26) has unique positive definite solution, and (6.24) converges to that solution.
- The closed-loop plant $(A - BK_\infty)$ is asymptotically stable.

The proof is ignored.

This implies that so long as the original system is stabilizable, the suboptimal control $u = -K_\infty x(k)$ guarantees the asymptotic stability of the system.

6.3. Continuous Optimal Control Basics

The dynamics of a continuous time domain system is described using differential equations, and the solution can be obtained using Lagrangian function similar to previous sections. However, do notice that calculus of variations is used when solving the problem. Calculus of variations is briefly reviewed in the appendix. For more details of calculus of variations, check *A Notebook on Calculus* by Lu Sun in the same series.

6.3.1. Problem Formulation

Let the system dynamics be described by the following state-space model

$$\dot{x} = f(x, u, t)$$

where $x(t) \in \mathbb{R}^n$, $u(t) \in \mathbb{R}^m$. Let the cost function be

$$J = \phi\left(x(T), T\right) + \int_0^T L\left(x(t), u(t), t\right) dt \tag{6.28}$$

where $[0, T]$ is the time interval of interest. The final time T may vary when different controls are applied. In addition, define $\psi\left(x(T), T\right)$. The objective of the optimization is to find optimal $u(t)$ on the time interval $[0, T]$, so that J in (6.28) is minimized, and $\psi\left(x(T), T\right) = 0$.

Notice that ψ is different from ϕ in (6.28). Both of them are about the final state, but ϕ is made small while ψ needs to be made exactly zero. This restriction introduced by *psi* is useful. For example, in a position regulation problem, ψ can be defined as

$$\psi = \begin{bmatrix} x(T) - X \\ \dot{x}(T) \end{bmatrix}$$

where $x(T) - X = 0$ enforces zero regulation error, and $\dot{x}(T) = 0$ enforces a complete stop of the system motion.

6.3.2. General Solution

Define the augmented cost function as the Lagrangian function below.

$$\begin{aligned} J' = {} & \phi\left(x(T), T\right) + \nu^T \psi\left(x(T), T\right) \\ & + \int_0^T \left(L(x, u, t) + \lambda^T \left(f(x, u, t) - \dot{x}\right)\right) dt \end{aligned}$$

where ν, λ are Lagrange multipliers. Define Hamiltonian function as follows.

$$H(x,u,t) \quad = \quad L(x,u,t) + \lambda^T f(x,u,t)$$

The solution of the optimization problem can be obtained by letting

State: $$\dot{x} = \frac{\partial H}{\partial \lambda} = f$$

Costate: $$-\dot{\lambda} = \frac{\partial H}{\partial x} = \frac{\partial f^T}{\partial x}\lambda + \frac{\partial L}{\partial x}$$

Stationarity: $$0 = \frac{\partial H}{\partial u} = \frac{\partial L}{\partial u} + \frac{\partial f^T}{\partial u}\lambda$$

Boundary: $x(0)$ is given

$$\left(\phi_x + \psi_x^T \nu - \lambda\right)^T \Big|_T dx(T) + \left(\phi_t + \psi_t^T \nu + H\right)^T \Big|_T dT = 0$$

It is often not easy to solve analytical solution of $u(t)$ from the above equations.

6.3.3. Continuous Linear Quadratic Regulator

LQR in discrete time domain has been introduced earlier. Its application in continuous time domain is introduced in this section as follows. Let the system dynamics be described by

$$\dot{x}(t) = A(t)x(t) + B(t)u(t)$$

where $x(t) \in \mathbb{R}^n$ and $u(t) \in \mathbb{R}^m$. The state vector $x(t)$ is assumed known exactly. The cost function is given by

$$J \quad = \quad \frac{1}{2}x(T)^T S_T x(T) + \frac{1}{2}\int_0^T \left(x(t)^T Q_t x(t) + u(t)^T R_t u(t)\right) dt$$

where $|S_T| \geq 0$, $|Q_t| \geq 0$ and $|R_t| > 0$ are known symmetric matrices.

The solution can be obtained by formulating the Hamiltonian. Result is given below.

$$-\dot{S}(t) \quad = \quad A(t)^T S(t) + S(t)A(t) - S(t)B(t)R_t^{-1}B(t)^T S(t) + Q_t \tag{6.29}$$

$$K(t) \quad = \quad R_t^{-1}B(t)^T S(t) \tag{6.30}$$

$$u(t) \quad = \quad -K(t)x(t)$$

6.3.4. Suboptimal Continuous Linear Quadratic Regulator

Just like Section 6.2.4, the continuous LQR also has suboptimal solution. The motivation remains the same: to tackle the problem where Kalman gain $K(t)$ in (6.30) is

time-varying and can be difficult or tedious to calculate, and we want an alternative solution with constant Kalman gain.

Assume time-invariant A, B in the process model and constant Q, R. Let time-invariant S be the solution of the following ARE

$$0 \quad = \quad A^T S + SA - SBR^{-1}B^T S + Q \tag{6.31}$$

which is obtained from (6.29) by letting $\dot{S} = 0$ assuming convergence. Notice that the solution to (6.31) is not necessarily unique. One of the solutions of S can be obtained by calculating (6.29) until it converges to $S(\infty)$, and $S = S(\infty)$ is obviously a solution to (6.31). This would be the solution that we would use through out the remaining of the section. Alternatively, S can be calculated using the analytical solution to the ARE which is not covered in this notebook.

With the obtained constant $S = S(\infty)$, Kalman gain in (6.30) becomes

$$K \quad = \quad R^{-1}B^T S(\infty)$$

which is time-invariant. This produces a suboptimal solution but with a constant Kalman gain that is easier for implementation.

6.4. Linear Quadratic Regulator Extensions

TBA

6.4.1. Safety Margin

The problem formulation and the solution of LQR have been introduced in earlier sections. The properties, extensions, practical implementations, etc., have not yet been covered. In this section and the few sections following, these more advanced topics are discussed.

In this section, we will start with the discussion of LQR properties, such as its robustness.

6.4.2. Tracking Problem

LQR by itself is a regulator that regulates the state vector x to a constant value r, and at the same time balancing the control signal energy. LQR can also be used in tracking, in which case the reference signal is no longer a constant, but time-variant. Tracking is an important extension of LQR.

6.5. Linear Quadratic Gaussian

TBA

6.6. H_2 and H_∞ Control

TBA

6.7. Numerical Optimal Control

TBA

7. Robust Control System

In a typical robust control problem, stochastic disturbance is introduced into the model, and such control signal that minimizes the average or worst effect of the disturbance is calculated. More details are introduced in this chapter.

8. Model Predictive Control System

Model predictive control system belongs to the broader family of optimal control system. It uses a model to forecast the system behavior, and computes such control sequence that optimizes the forecast while fulfilling all constraints. The optimization problem is solved recurrently at each timestamp, with only the first control from the optimal sequence actually applied.

Due to its popularity, model predictive control system is introduced in this dedicated chapter but not in Chapter 6.

8.1. Introduction

A **model predictive control** (MPC) system uses a dynamic model to predict the behavior of the plant, and then find the control sequence that minimizes the cost function formulated from the forecast to produce the best decision, meanwhile fulfilling all the constraints applied to the state and control.

MPC is often compared with LQR which has been introduced in Chapter 6. Indeed, they are similar to some extent. For instance, they are both parametric model based optimization control. MPC is often regarded as a more robust and flexible solution than LQR, but with a higher computation burden. Their differences will be discussed in detail in the remainder of the chapter.

The following information is required to calculate the optimal control sequence:

- The plant, which is used to forecast the system trajectory
  - Model
  - Current state
  - Constraints for both state and control signal
- Cost function

In practice, the model of the plant is usually calibrated by parameter estimation and the current state by state estimation. The constraints can be obtained from the equipment handbook. The cost function, usually in the form of a functional, is defined by the user.

8.1.1. Model and Constraint

The model plays an absolutely critical role in MPC. A model can be either linear or nonlinear, continuous or discrete, in state-space or input-output realizations, centralized or discrete (not spatially uniform), deterministic or stochastic. Different problem

formulations and mathematical tools are used to describe the different types of the models and solve the different types of optimization problems.

MPC has a good adaption to different types of constraints. The constraints can be largely divided into two categories, namely physical constraints and performance constraints. Input signal $u(k)$ related constraints are often physical constraints reflecting the physical limits of the system. State vector $x(k)$ and output $y(k)$ related constraints, on the other hand, are often performance constraints. The performance constraints describes the lowest performance goals that the user expects from the system.

Input signal $u(k)$ related constraints are often given in the following form.

$$\begin{bmatrix} -I \\ I \end{bmatrix} u(k) \quad \leq \quad \begin{bmatrix} -u_{\min} \\ u_{\max} \end{bmatrix} \tag{8.1}$$

State vector $x(k)$ related constraints are often given in the following form.

$$Fx(k) \quad \leq \quad f \tag{8.2}$$

It is possible to use augmented state vector that combines states with inputs. Let

$$\tilde{x}(k) \quad = \quad \begin{bmatrix} x(k) \\ u(k-1) \end{bmatrix}$$

and both constraints in (8.1) and (8.2) can be expressed by the form

$$F\tilde{x}(k) + Eu(k) \quad \leq \quad e \tag{8.3}$$

Equation (8.3) can also be used to limit the difference signal of the control signal. Consider

$$\begin{bmatrix} 0 & -I \\ 0 & I \end{bmatrix} \tilde{x}(k) + \begin{bmatrix} I \\ -I \end{bmatrix} u(k) \quad \leq \quad \begin{bmatrix} \Delta_{\max} \\ -\Delta_{\min} \end{bmatrix}$$

in (8.3), which essentially translates to

$$\Delta_{\min} \leq u(k) - u(k-1) \leq \Delta_{\max}$$

Other constraints include limiting the state vector and/or the control signal to be discrete values such as integers. These types of constraints are not as commonly seen as the aforementioned ones.

8.1.2. Problem Formulation

The optimal control to the plant vary with the definition of the cost function and constraints. In this section, for simplicity, a linear quadratic regulator problem is used to demonstrate MPC. Notice that although LQR is often regarded as a stand alone optimal control problem, there is no harm to introduce LQR as a simplified MPC problem, just for demonstration purpose.

Finite LQR Formulation

Consider the following dynamic model in discrete time domain. Current timestamp is $k = 0$. For simplicity, it is assumed that the model is known and the state is measurable precisely, thus there is no need to build an estimator for the model or the state.

$$\begin{aligned} x(k+1) &= Ax(k) + Bu(k) && (8.4) \\ y(k) &= x(k) && (8.5) \end{aligned}$$

With the above model, it is possible to predict how the state evolves given a sequence of inputs as below.

$$u = \begin{bmatrix} u(0) & u(1) & \dots & u(N-1) \end{bmatrix}$$

where N is the largest time scale considered in this problem.

The purpose of the control is to regulate non-zero initial state $x(0)$ to zero while keeping the magnitude of $u(k)$ small. Therefore, define the cost function as follows.

$$V(x(0), u) = \frac{1}{2} \sum_{k=0}^{N-1} \left(x(k)^T Q x(k) + u(k)^T R u(k) \right) + \frac{1}{2} x(N)^T P_f x(N) \tag{8.6}$$

where $Q \geq 0$, $P_f \geq 0$ and $R > 0$ are symmetric matrices of user's choice. It can be proved that the above problem formulation must have a unique solution.

A Brief Introduction to Dynamic Programming

The aforementioned optimization problem in (8.6) can be solved via variety of ways. Consider using dynamic programming (DP).

A brief introduction to DP is given as follows. For illustration of DP, consider the following simple optimization problem.

$$x^*, y^* = \arg\min_{x,y} f(x, y) \tag{8.7}$$

Since x, y are coupled in $f(x, y)$, an intuitive way of solving this optimization problem is to let

$$\begin{cases} \dfrac{\partial}{\partial x} f(x, y) = 0 \\ \dfrac{\partial}{\partial y} f(x, y) = 0 \end{cases}$$

and if $f(x, y)$ is quadratics of x, y, the above should generate a unique solution.

Alternatively, consider solving (8.7) using the following approach. Think of (8.7) as a two-stage optimization problem. Given any constant value of y, there is an associated optimized $x^*(y)$ which minimizes (8.7). Using this method, we can transform $f(x, y)$

into $f\left(x^*(y), y\right)$, a function of y alone. From there, the optimal y can be found easily. Subsequently, $x^*(y)$ can be obtained as well. To summarize, (8.7) is equivalent of

$$\begin{aligned} y^* &= \arg\min_y \left(\min_x f(x,y)\right) \\ x^* &= \arg\min_x f(x,y^*) \end{aligned}$$

Solving using DP as above can simplifies the problem, especially When only part of the optimal variable values is required (for example, only y^* is required).

Solution to the Finite LQR Problem

The LQR problem in (8.6) is essentially something similar with (8.7). The initial state $x(0)$ is a constant in the optimization problem, the control signals $u(0), ..., u(N-1)$ to be optimized, and the coupling introduced by the system dynamics (8.4). The ultimate goal of the optimization problem is to determine $u(0)$.

Similarly, the optimization is divided into multiple stages, where in each stage only one instant $u(k)$ is considered, and other variables leading to that stage is considered as constant. We start with the last control input $u(N-1)$. The optimization problem is formulated as follows.

$$\begin{aligned} V^{\text{sub}}(N-1) &= \frac{1}{2}\left(x(N-1)^T Q x(N-1) + u(N-1)^T R u(N-1)\right) \\ &\quad + \frac{1}{2} x(N)^T P_f x(N) \\ \text{s.t.} &\quad x(N) = Ax(N-1) + Bu(N-1) \end{aligned}$$

where $x(N-1)$ is taken as a constant from another stage.

Solving $u(N-1)$ as a function of $x(N-1)$ gives

$$\begin{aligned} u^*(N-1) &= K_{N-1} x(N-1) \\ x(N) &= (A + BK_{N-1}) x(N-1) \\ V^{\text{sub}}(N-1) &= \frac{1}{2} x(N-1)^T \Pi_{N-1} x(N-1) \end{aligned}$$

where K_{N-1} and Π_{N-1} are two intermediate parameters calculated as follows.

$$\begin{aligned} K_{N-1} &= -(B^T P_f B + R)^{-1} B^T P_f A \\ \Pi_{N-1} &= Q + A^T P_f A - A^T P_f B (B^T P_f B + R)^{-1} B^T P_f A \end{aligned}$$

Consider the next stage optimization problem as follows.

$$\begin{aligned} V^{\text{sub}}(N-2) &= \frac{1}{2}\left(x(N-2)^T Q x(N-2) + u(N-2)^T R u(N-2)\right) \\ \text{s.t.} &\quad x(N-1) = Ax(N-2) + Bu(N-2) \end{aligned}$$

where $x(N-2)$ is taken as constant and $u^*(N-2)$ is to be found. The solution is given by

$$\begin{aligned} u^*(N-2) &= K_{N-2}x(N-2) \\ x(N-1) &= (A+BK_{N-2})x(N-2) \\ V^{\text{sub}}(N-2) &= \frac{1}{2}x(N-2)^T\Pi_{N-2}x(N-2) \end{aligned}$$

where

$$\begin{aligned} K_{N-2} &= -(B^T\Pi_{N-1}B+R)^{-1}B^T\Pi_{N-1}A \\ \Pi_{N-1} &= Q+A^T\Pi_{N-1}A-A^T\Pi_{N-1}B(B^T\Pi_{N-1}B+R)^{-1}B^T\Pi_{N-1}A \end{aligned}$$

It is clear by now that the above calculations can be done recursively to obtain $u^*(0)$. Notice that all the control signals have the following form

$$u^*(k) = K_k x(k)$$

which implies that the optimal control can be taken as a classic state-space feedback controller with time-varying control gains.

Infinite LQR Problem

Since the recursive calculation is done N times to calculate $K_{N-1}, ..., K_0$, apparently different value of N results in different K_0. It is worth mentioning that optimal control does not suggest stable control. In other words, it is possible for a specific system plant, specific choice of Q, R, P_f, and specific N, to lead to an unstable closed loop system, i.e., $A+BK_0$ with eigenvalues larger than 1. However, it is guaranteed that when $N \to \infty$, K_0 converges, and $A+BK_0$ is always stable. This introduces the infinite LQR problem, where the cost function corresponding with (8.6) is changed to

$$V(x(0),u) = \frac{1}{2}\sum_{k=0}^{\infty}\left(x(k)^TQx(k)+u(k)^TRu(k)\right)$$

Infinite LQR can be taken as a special case of finite LQR where $N \to \infty$.

We have been making an underlying assumption that the original plant is controllable. This means that the system can be driven from any initial state to $x=0$ in finite time. Apply this concept to infinite LQR problem. This implies that in the beginning finite steps of this infinite control sequence, the state should have already been regulated to zero.

Since the control gain $K(k), k = N-1, N-2, ...$ converges to K, the first infinite control gains would be K. Therefore, it can be concluded that the infinite LQR suggests the control gain to remain K until all states becomes zero. Therefore, the infinite LQR problem results in constant control gain rather than time-varying control gains as in the finite LQR problem. The control gain can be obtained by solving the Riccati equation of Π as follows.

$$\begin{aligned} K &= -(B^T\Pi B+R)^{-1}B^T\Pi A \\ \Pi &= Q+A^T\Pi A-A^T\Pi B(B^T\Pi B+R)^{-1}B^T\Pi A \end{aligned}$$

8.1.3. Control Objectives

The most commonly seen MPC objectives are tracking, state regulation, and disturbance rejection.

- Tracking: the output of the plant shall follow the given reference point (not necessarily to be a constant) as close as possible.
- State regulation: the state vector of the plant shall converge to a given steady-state as soon as possible, while not violating system restrictions.
- Disturbance rejection: the plant shall maintain its operating point under unknown stochastic noise and disturbances.

8.2. State-Space Model Predictive Control

TBA

8.3. Input-Output Model Predictive Control

TBA

8.4. Robust Model Predictive Control

TBA

8.5. Distributed Model Predictive Control

TBA

9. Adaptive Control System

Adaptive controller is generally referred to a controller with adjustable parameters and associated mechanism to adjust such parameters, to conform to new circumstances.

Comparing with most of the other control systems, an adaptive control system does not require a very precise modeling of the plant. More precisely, an adaptive control system can adapt the control gains to the unknown or time-varying plant and maintain the stability of the closed-loop system.

The references of this chapter include:

- Åström, K.J. and Wittenmark, B., 2013. Adaptive control. Courier Corporation [1].

9.1. Introduction

Adaptive controller is generally referred to a controller with adjustable parameters and associated mechanism to adjust such parameters, to conform to new circumstances.

Due to the parameter adjustment mechanism, the controller becomes non-linear. For example, while a conventional PID controller is linear, an adaptive PID controller is nonlinear since the $P(t)$, $I(t)$ and $D(t)$ are variables instead of constant gains.

In practice, an adaptive controller is often a modified version of a conventional controller just like the adaptive PID controller example. There are often 2 types of feedback loops in an adaptive control system: the original loops that comes with the conventional controller, and the additional loops to adjust the parameters of the conventional controller. A demonstrative figure is given in Fig. 9.1.

9.1.1. A Brief History of Adaptive Control System

The first adaptive control system application was designed for autopilots of a high-performance aircraft, where the conventional control system was found to work for one condition but not for the other. Gain scheduling was adopted to solve this problem.

Later on more sophisticated adaptive control systems that utilizes state-space model and stability theory were introduced. Stochastic control theory and system identification techniques were developed side-by-side.

The proof of stability of the adaptive control system, especially universal stability of the system, started to draw attention. The proof can be done often only with strong restrictions. People started to investigate the connection and difference of robust control and system identification with adaptive control and system identification. Computer controlled system and artificial neural network started to emerge and people realized the connection between adaptive control and computer learning.

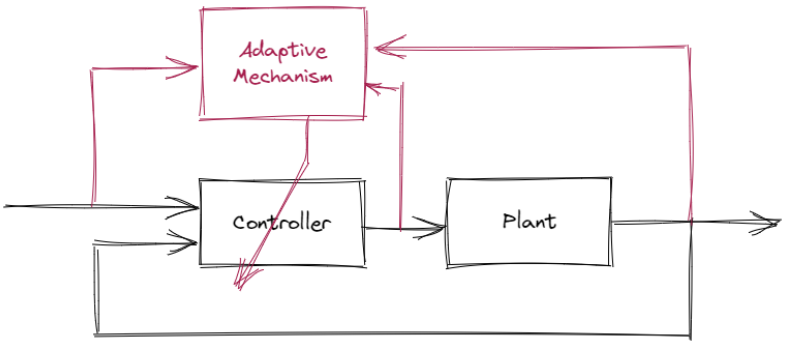


Figure 9.1.: Adaptive control system general schema.

Adaptive control system commercialization started in 1980s, and are used in handling process dynamics and disturbances, and to provide automatic tuning of controller parameters.

9.1.2. Conventional Control System

The most conventional way of designing a control system is as follows.

1. Assume a linear model of the plant at the operating point.
2. Calculate or estimate the parameters of the plant model.
3. Design a closed-loop control system for the plant.

The control system designed following the above approach is usually intrinsically insensitive to modeling error and disturbance to some extend (due to the closed-loop implementation). But there are difficulties that could cause variation and troubles.

- **Nonlinear actuator** is one of the sources of modeling uncertainty, making the control system work only on/near the pre-determined operating point, but not universally.
- **Flow and speed variations** in the pipes of a system, if exist, can change from time to time. The different flow and speed can cause changes in the process dynamics, thus change the plant model.
- **Environmental dynamics and uncertainties** often draw significant concerns in control. For example, the aircraft flying at different height and speed suffers from different environmental conditions. The same applies to ship steering at different speed.

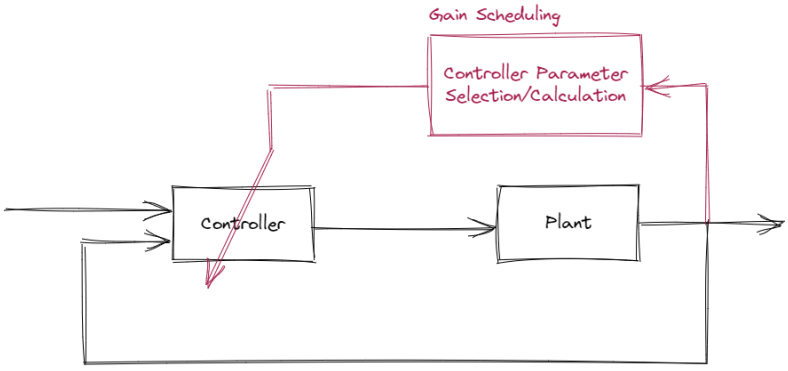


Figure 9.2.: Gain scheduling schema.

9.1.3. Adaptive Control Schema

Different adaptive control schemes have been designed to tackle the above issues. The commonly used ones are briefly introduced here.

Gain Scheduling

Measurement feedback may correlate well with changes in the process dynamics. The idea of gain scheduling is to map the process dynamic parameters with the control parameters, by either a function manner or a table lookup manner, i.e., schedule the control parameters to compensate the the process dynamics.

A demonstrative figure is given in Fig. 9.2.

Gain scheduling is one of the most intuitive adaptive control schema, and has the earliest use case among all.

Model-Reference Adaptive System (MRAS)

In this scenario, a reference model is built that generates the "ideal" output of the closed-loop system given the reference point. This is often a virtual model that does not suffer from disturbances and error.

The MRAS then uses the output of both the actual plant and the reference model to calculate the controller parameter adjustment mechanism to minimize the difference between the outputs of the two models.

A demonstrative figure is given in Fig. 9.3.

The MRAS is somewhat like a learning system. Usually, there is a "adaptation rate" (just like the learning rate) that controls the adapting speed.

Self-Tuning Regulator (STR)

The idea of STR is to estimate the system process and design and change the whole control system in real-time. Unlike the gain scheduling approach and MRAS where parameter adjustment mechanism is calculated in real-time, in STR approach, the

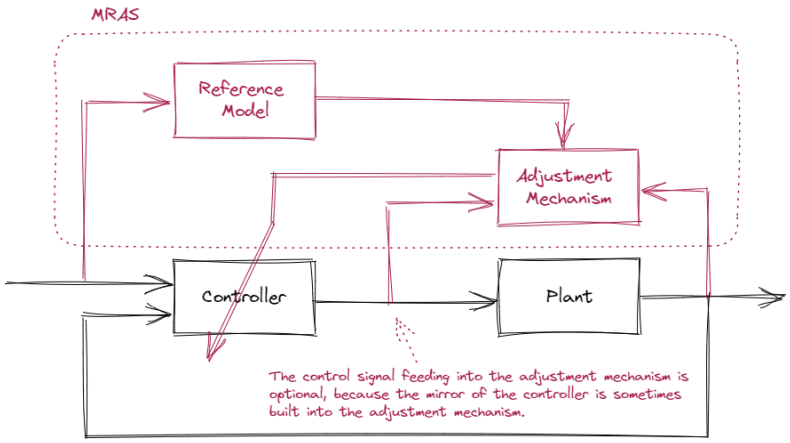


Figure 9.3.: MARS schema.

controller designing problem is solved in real-time. From this sense, STR can be taken as the "high-end" gain scheduling problem.

A demonstrative figure is given in Fig. 9.4.

STR can be made very flexible, because it is essentially a new control system design from scratch each time the environment changes.

Dual Control

The aforementioned adaptive control schemas do not taken into account the stochastics and statistics of the measurement and estimation uncertainty.

To analyze the performance of the system under estimation uncertainty and to develop control algorithm that can optimize the system performance under such uncertainty, nonlinear stochastic control theory needs to be used, which leads to the notion of dual control.

A demonstrative figure is given in Fig. 9.5.

This control system is essentially a nonlinear controller with a robust/adaptive filter that estimates the process dynamics and the states of the plant, together with statistics information of the estimates.

9.1.4. A Typical Adaptive Control Problem Formulation

For simplicity, consider the following LTI plant realizations.

Continuous time domain state space model:

$$\begin{aligned} \dot{x}(t) &= Ax(t) + Bu(t) \\ y(t) &= Cx(t) \end{aligned}$$

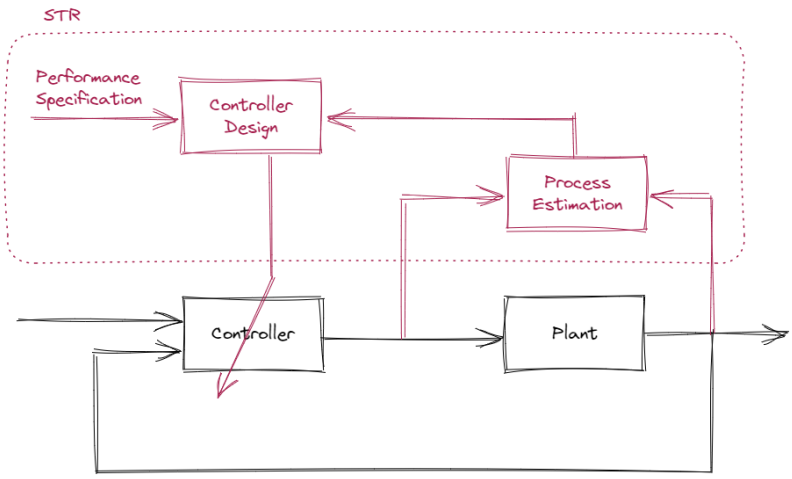


Figure 9.4.: STR schema.

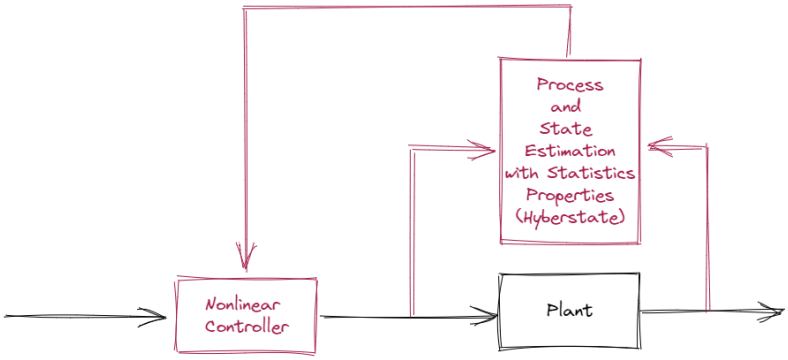


Figure 9.5.: Dual control schema.

Continuous time domain transfer function model:

$$G(s) = \frac{B(s)}{A(s)} = \frac{b_0 s^m + b_1 s^{m-1} + \ldots + b_m}{s^n + a_1 s^{n-1} + \ldots + a_n}$$

Continuous time domain input-output model (with p the differential operator, $p = \frac{d}{dt}$):

$$y(t) \quad = \quad G(p)u(t)$$

Discrete time domain state-space model:

$$\begin{aligned} x(k+1) &= Ax(k) + Bu(k) \\ y(k) &= Cx(k) \end{aligned}$$

Discrete time domain transfer function:

$$G(z) = \frac{B(z)}{A(z)} = \frac{b_0 z^m + b_q z^{m-1} + \ldots + b_m}{z^n + a_1 z^{n-1} + \ldots + a_n}$$

Discrete time domain input-output model (with q the forward shift operator, $qy(k) = y(k+1)$):

$$y(k) \quad = \quad H(q)u(k)$$

The above models may be describing the same plant, with different level of details. For example, when dealing with transfer function and input-output models, the initial condition of the system is often ignored. When dealing with discrete time domain plant model, it is open a sampled system derived from the original system.

If the parameters in the models are known, it is straight forward to design a closed-loop controller for them. This is called the underlying design problem. The adaptive control problem, on the other hand, is to design a controller without knowing the parameters. In response, the controller comes with adjustable control gains and associated adjustment mechanism.

A typical adaptive controller design shall include the following procedures:

1. Characterize the desired closed-loop behavior.
2. Determine a suitable control law with adjustable parameters.
3. Design adjustment mechanism.
4. Implement the control law with the parameters adjusted using the adjustment mechanism.

Some successful use cases for adaptive control systems include

- Automatic tuning controller, such as the PID automatic tuning controller. In practice, all conventional control systems can benefit from automatic parameter tuning, if done correctly.

- Gain scheduling, which is the standard technique used in flight control system for high-performance aircraft, and is becoming increasingly popular in industrial process control.
- Continuous adaptation control for continuously varying system.
- "Human-in-the-loop" control system.

Since adaptive control system is often more costly and complicated than a conventional constant-gain controller, it should be used wisely and not abused. If the dynamics of the system are manageable, consider using parametric model based control algorithm rather than the adaptive control system, the later of which is usually more complicated.

9.2. Parameter Estimation

Parameter estimation (or more generally, system identification) is widely used, either directly or indirectly, in many adaptive control schemas. An obvious example is STR, where the process dynamics estimation is built into the controller. Parameter estimation also happens implicitly in MRAS.

The key factors in system identification include the following.

- Selection of model structure.
- Experiment design.
- Parameter and state estimation.
- Validation.

One thing to notice is that in adaptive control scope, the plant is assumed to change continuously, thus the parameter estimation needs to be done in a recursive manner.

9.2.1. Least Squares Estimation

Karl Friedrich Gauss defines LS estimation problem as follows.

LS estimation:

The parameters in the model should be chose such that the sum of the squares of the differences between the observed and the computed values, multiplied by numbers that measure the degree of precision, is a minimum.

Notice that from the above definition, if all observations share the same degree of precision, the multipliers to all observations shall be the same, otherwise they should be different. Conventionally, the later case is referred as "Weighted LS (WLS) estimation" where each multiplier is called the "weight" associated with the observation. The more

precise an observation, the higher its associated weight. On the other hand, the former case where all observations share the same precision is referred as LS estimation.

To reduce confusion, from now on LS estimation and WLS estimation are used to refer to the former and later cases, respectively.

Formulate LS estimation problem on a LTI system as follows. Let $\theta_i, i = 1, ..., n$ be n unknown parameters in the model and $y(j), j = 1, ..., m$ be m measurements collected from the mode. It is required that $m \geq n$. Variable $\phi_i(j)$ is called a regression variable (or a regressor) that links the measurements with the model parameters, so that

$$y(j) = \phi_1(j)\theta_1 + \phi_2(j)\theta_2 + \ldots + \phi_n(j)\theta_n + \epsilon(j) \tag{9.1}$$

where $\epsilon(j)$ is assumed to be independent measurement noise associated with $y(j)$. Equation (9.1) is called a regression model. Putting (9.1) into matrix form gives

$$y = \Phi\theta + \epsilon \tag{9.2}$$

where

$$\begin{aligned} y &= \begin{bmatrix} y(1) & \ldots & y(m) \end{bmatrix}^T \\ \Phi &= \begin{bmatrix} \phi_1(1) & \ldots & \phi_n(1) \\ \vdots & \ddots & \vdots \\ \phi_1(m) & \ldots & \phi_n(m) \end{bmatrix} \\ \theta &= \begin{bmatrix} \theta_1 & \ldots & \theta_n \end{bmatrix}^T \\ \epsilon &= \begin{bmatrix} \epsilon(1) & \ldots & \epsilon(m) \end{bmatrix}^T \end{aligned}$$

with y the measurement vector, Φ the regression model matrix, and θ the unknown system parameter vector.

LS estimation of θ in (9.2) is formulated as follows. Select estimation $\hat{\theta} = [\hat{\theta}_1, ..., \hat{\theta}_n]^T$ so that the following cost function $V(\hat{\theta})$ is minimized.

$$V(\hat{\theta}) = \frac{1}{2}\sum_{i=1}^{m} e(i)^2 \tag{9.3}$$

where

$$\begin{aligned} e(i) &= y(i) - \left(\phi_1(i)\hat{\theta}_1 + \phi_2(i)\hat{\theta}_2 + \ldots + \phi_n(i)\hat{\theta}_n\right) \\ &= y(i) - \phi(i)^T\hat{\theta} \end{aligned} \tag{9.4}$$

with

$$\phi(i)^T = \begin{bmatrix} \phi_1(i) & \ldots & \phi_n(i) \end{bmatrix} \tag{9.5}$$

which is the ith row of Φ. Variable $e(i)$ is called the residual.

Put everything into matrix format. Equation (9.3) becomes

$$\begin{aligned} V(\hat{\theta}) &= \frac{1}{2}\sum_{i=1}^{m}\left(y(i)-\phi(i)^T\hat{\theta}\right)^2 \\ &= \frac{1}{2}(y-\Phi\hat{\theta})^T(y-\Phi\hat{\theta}) \end{aligned} \tag{9.6}$$

Denote $e=[e(1),...,e(m)]^T$. From (9.4),

$$e = y-\Phi\hat{\theta} \tag{9.7}$$

and (9.6) becomes

$$V(\hat{\theta}) = \frac{1}{2}e^Te \tag{9.8}$$

The quadratic optimization problem given in (9.8) has the following unique analytical solution. Differentiating (9.8) w.r.t. θ gives (using numerator-layout notation)

$$\frac{dV(\hat{\theta})}{d\theta} = \frac{dV(\hat{\theta})}{de}\frac{de}{d\theta} \tag{9.9}$$

$$= -e^T\Phi \tag{9.10}$$

Equating (9.10) to zero gives the minimum of (9.8) as follows.

$$-e^T\Phi = 0 \tag{9.11}$$

Substituting (9.7) into (9.11) gives

$$\begin{aligned} -\left(y-\Phi\hat{\theta}\right)^T\Phi &= 0 \\ -\Phi^T\left(y-\Phi\hat{\theta}\right) &= 0 \\ \Phi^T\Phi\hat{\theta} &= \Phi^Ty \\ \hat{\theta} &= \left(\Phi^T\Phi\right)^{-1}\Phi^Ty \end{aligned} \tag{9.12}$$

provided that $\Phi^T\Phi$ is nonsingular, which is known as the excitation condition. Equation (9.12) is the solution to the LS estimation problem in (9.3).

A Quick Note on Matrix Differentiation

In numerator-layout notion, the scalar $V(\hat{\theta})$ differentiated w.r.t. the vector e in (9.9) is given by

$$\begin{aligned} \frac{dV(\hat{\theta})}{de} &= \left[\begin{array}{ccc} \frac{dV(\hat{\theta})}{de(1)} & \cdots & \frac{dV(\hat{\theta})}{de(m)} \end{array}\right] \\ &= \left[\begin{array}{ccc} e(1) & \cdots & e(m) \end{array}\right] \\ &= e^T \end{aligned}$$

The vector e differentiated w.r.t. the vector θ is given by

$$\begin{aligned}\frac{de}{d\theta} &= \begin{bmatrix} \frac{de(1)}{d\theta_1} & \cdots & \frac{de(m)}{d\theta_n} \\ \vdots & \ddots & \vdots \\ \frac{de(m)}{d\theta_1} & \cdots & \frac{de(m)}{d\theta_n} \end{bmatrix} \\ &= \begin{bmatrix} -\phi_1(1) & \cdots & -\phi_n(1) \\ \vdots & \ddots & \vdots \\ -\phi_1(m) & \cdots & -\phi_n(m) \end{bmatrix} \\ &= -\Phi\end{aligned}$$

where

$$\frac{de(i)}{d\theta_j} = -\phi_j(i)$$

because of (9.4).

Therefore,

$$\frac{dV(\hat{\theta})}{d\theta} = \frac{dV(\hat{\theta})}{de}\frac{de}{d\theta} = -e^T\Phi$$

which is (9.10).

A WLS estimation problem can be formulated similarly, by changing (9.3) with

$$V(\hat{\theta}) = \frac{1}{2}\sum_{i=1}^{m} w(i)e(i)^2$$

The solution to a WLS estimation can be similarly derived and the result is

$$\hat{\theta} = \left(\Phi^T W \Phi\right)^{-1}\Phi^T W y$$

where $W = \mathrm{diag}(w(1), ..., w(m))$.

9.2.2. Statistics Properties of LS Estimation

In the discussions below, We will assume that all measurement noise $\epsilon(i), i = 1, ..., m$ are i.i.d. noise, whose mean and variance are zero and σ^2 respectively.

Substituting (9.2) into (9.12) gives

$$\begin{aligned}\hat{\theta} &= \left(\Phi^T\Phi\right)^{-1}\Phi^T\left(\Phi\theta + \epsilon\right) \\ &= \theta + \left(\Phi^T\Phi\right)^{-1}\Phi^T\epsilon \qquad (9.13)\end{aligned}$$

Using (9.13), the following statistics properties of LS estimation can be drawn.

$$\begin{aligned} E\left[\hat{\theta}\right] &= \theta \\ \text{Cov}\left[\hat{\theta}\right] &= E\left[\hat{\theta}\hat{\theta}^T\right] \\ &= \sigma^2\left(\Phi^T\Phi\right)^{-1} \end{aligned}$$

which suggests that the estimator is zero mean. With the fixed number n of parameters to be estimated, the estimation variance decreases as the number m of observations increases, and the variance converges to zero eventually. The variance of the noise σ^2 can potentially be estimated from the state estimate $\hat{\theta}$. Details are given as follows. Substituting (9.2), (9.12) into (9.7) gives

$$e = \left(I - \Phi\left(\Phi^T\Phi\right)^{-1}\Phi^T\right)\epsilon \tag{9.14}$$

Using (9.14),

$$E\left[ee^T\right] = \sigma^2\left(I - \Phi\left(\Phi^T\Phi\right)^{-1}\Phi^T\right)\left(I - \Phi\left(\Phi^T\Phi\right)^{-1}\Phi^T\right)^T \tag{9.15}$$

Taking the trace of matrix from both sides of (9.15) gives

$$E\left[e^Te\right] = (m-n)\sigma^2 \tag{9.16}$$

From (9.8) and (9.16),

$$\sigma^2 = \frac{2}{m-n}E\left[V(\hat{\theta})\right]$$

can be used to estimate the variance of the noise.

9.2.3. Recursive LS Estimation

Recursive state estimation allows "updating" the state estimate each time a new measurement set becomes available. It takes advantages of previously calculated state estimate with less measurements, instead of calculating everything from scratch, which is also known as the batch estimation. Recursive estimation saves computational and memory burden than the batch estimation. Given that observations are often naturally obtained sequentially in real time, recursive estimation has a large demand in practice.

Let each measurement be obtained associated with a incremental time stamp $k = 1, 2, ...$ While the number of parameters to be estimated remains constant n, the number of measurement m is dynamic and replaced by the notion k.

It is worth mentioning that the size of Φ in (9.2) grows with k, in the sense that each time a new measurement becomes available, Φ grows by one row. For convenience, $\Phi(k)$ is used to describe Φ in (9.2) when there are k measurements.

As more and more measurements come in, the state estimate $\hat{\theta}$ also evolves and become more and more accurate. Denote $\hat{\theta}(k)$ the state estimate when up to k measurements are available, $k \geq n$.

In recursive estimation schema, the estimator is formulated as a Markov process, and the state estimate $\hat{\theta}(k+1)$ is formulated as a function of $\hat{\theta}(k)$ and $y(k+1)$, and does not directly involve the earlier state estimates $\hat{\theta}(k-1), \hat{\theta}(k-2), ...$ or measurements $y(k), y(k-1), ...$ This is possible because the information of $\hat{\theta}(k-1), \hat{\theta}(k-2), ...$ and $y(k), y(k-1), ...$ are integrated into $\hat{\theta}(k)$.

The derivation of the recursive estimation is as follows. Notice that $\phi(i)^T$ in (9.5) is the ith row of $\Phi(k)$, hence

$$\Phi(k)^T\Phi(k) \quad = \quad \sum_{i=1}^{k} \phi(i)\phi(i)^T \tag{9.17}$$

Assume that the excitation condition holds. Denote $P(k) = \left(\Phi(k)^T\Phi(k)\right)^{-1}$, or equivalently $P(k)^{-1} = \Phi(k)^T\Phi(k)$. Use (9.17),

$$P(k+1)^{-1} \quad = \quad P(k)^{-1} + \phi(k+1)\phi(k+1)^T \tag{9.18}$$

which can be calculated recursively.

Rewrite (9.12) as follows.

$$\begin{aligned}
\hat{\theta}(k) &= \left(\sum_{i=1}^{k} \phi(i)\phi(i)^T\right)^{-1} \sum_{i=1}^{k} \phi(i)y(i) \\
&= P(k) \sum_{i=1}^{k} \phi(i)y(i) \\
\hat{\theta}(k+1) &= \left(\sum_{i=1}^{k+1} \phi(i)\phi(i)^T\right)^{-1} \sum_{i=1}^{k+1} \phi(i)y(i) \\
&= P(k+1)\left(\sum_{i=1}^{k} \phi(i)y(i) + \phi(k+1)y(k+1)\right) \\
&= P(k+1)\left(P(k)^{-1}\hat{\theta}(k) + \phi(k+1)y(k+1)\right) \\
&= P(k+1)\Big(\left(P(k+1)^{-1} - \phi(k+1)\phi(k+1)^T\right)\hat{\theta}(k) \\
&\quad +\phi(k+1)y(k+1)\Big) \\
&= \hat{\theta}(k) \\
&\quad +P(k+1)\phi(k+1)\left(y(k+1) - \phi(k+1)^T\hat{\theta}(k)\right)
\end{aligned} \tag{9.19}$$

Equation (9.18) and (9.19) give the recursive LS estimation. Notice that the (9.18) gives the recursive calculation of $P(k)^{-1}$. The recursive calculation of $P(k)$ can be

easily derived from (9.18) using matrix inversion lemma as follows.

$$\begin{aligned}
P(k+1) &= P(k) - P(k)\phi(k+1) \\
&\quad \times \left(I + \phi(k+1)^T P(k)\phi(k+1)\right)^{-1} \phi(k+1)^T P(k) \\
P(k+1)\phi(k+1) &= P(k)\phi(k+1) \\
&\quad \times \Big(I - \left(I + \phi(k+1)^T P(k)\phi(k+1)\right)^{-1} \\
&\quad \times \phi(k+1)^T P(k)\phi(k+1)\Big) \\
&= P(k)\phi(k+1)\left(I + \phi(k+1)^T P(k)\phi(k+1)\right)^{-1} \\
&\quad \times \left(\left(I + \phi(k+1)^T P(k)\phi(k+1)\right)\right. \\
&\quad \left. -\phi(k+1)^T P(k)\phi(k+1)\right) \\
&= P(k)\phi(k+1)\left(I + \phi(k+1)^T P(k)\phi(k+1)\right)^{-1}
\end{aligned}$$

where notice that in the above equations the identity matrix I under the scope of our discussion should be 1, as the measurement at any instant, $y(k)$, is a scalar. We followed the convention using I because in practice, the measurement at any instant can be a vector. Hopefully this explanation helps to clear some confusion.

Denote $K(k) = P(k)\phi(k)$ the Kalman gain, and residual $\varepsilon(k+1) = y(k+1) - \phi(k+1)^T\hat{\theta}(x)$ the innovation vector, where $\phi(k+1)^T\hat{\theta}(x)$ can be interpreted as a "guess" of $y(k+1)$ using the state estimate $\hat{\theta}(x)$. Recursive LS estimation can be taken as a special case of Kalman filter where (a) measurement $y(k)$ is a scalar; (b) process matrix is unit matrix; (c) input and process noise are both zero.

9.2.4. LS Estimation with Time-Varying Parameters

If the parameter is time varying and has a solid dynamic model, considering using Kalman filter and other parametric model based estimators. If not, consider two scenarios: (a) parameters changes abruptly and infrequently; (b) parameters changes slowly and continuously.

Consider case (a). In this case, the abrupt changes can be detected from the magnitude of the innovation vector. The innovation vector covariance is jointly decided by the measurement noise and the state estimate precision. When the parameter is constant, the covariance matrix converges to a certain value that can be calculated analytically. A simple test can be used to detect when the innovation vector violates the statistics assumption, indicating that there is an abrupt parameters change.

In such case, just "reset" the system by assuming the new measurement is the first set of measurement collected by the system. In practice, this can be achieved equivalently by reset $P(k-1)$ and/or $P(k)$ to a diagonal matrix with large elements.

Consider case (b). One way to handle this parameter change is to use moving horizon estimation, or introducing a forgetting factor into the cost function. An example of

introducing forgetting factor is given as follows. In (9.6), set

$$V(\hat{\theta}) = \frac{1}{2}\sum_{i=1}^{k}\lambda^{k-i}\left(y(i)-\phi(i)^T\hat{\theta}\right)^2 \tag{9.20}$$

where $0 < \lambda < 1$ is a forgetting factor, and the method given in (9.20) is called exponential forgetting. By introducing the forgetting factor, the contribution of innovation introduced by early measurements and state estimates to the cost function is reduced, thus reducing their impact on the state estimates.

The batch and recursive algorithms to calculate the state estimates for (9.20) can be derived similarly. Some key derivations are given below.

Equation (9.12) becomes

$$\begin{aligned}\hat{\theta} &= \left(\Phi^T\Lambda\Phi\right)^{-1}\Phi^T\Lambda y\\ \Lambda &= \mathrm{diag}\begin{pmatrix}\lambda^{m-1} & \dots & \lambda & 1\end{pmatrix}\end{aligned}$$

Equation (9.18) becomes

$$\begin{aligned}P(k)^{-1} &= \Phi(k)^T\Lambda\Phi(k)\\ P(k+1)^{-1} &= \lambda P(k)^{-1}+\phi(k+1)\phi(k)^T\end{aligned}$$

Applying matrix inversion lemma to the above equation gives

$$\begin{aligned}P(k+1) &= \frac{1}{\lambda}P(k)-\frac{1}{\lambda}P(k)\phi(k+1)\\ &\quad\times\left(\lambda I+\phi(k+1)^TP(k)\phi(k+1)\right)^{-1}\phi(k+1)^TP(k)\\ K(k+1) &= P(k+1)\phi(k+1)\\ &= P(k)\phi(k+1)\left(\lambda I+\phi(k+1)^TP(k)\phi(k+1)\right)^{-1}\end{aligned}$$

Equation (9.19) has the same eventual form of

$$\hat{\theta}(k+1) = \hat{\theta}(k)+P(k+1)\phi(k+1)\left(y(k+1)-\phi(k+1)^T\hat{\theta}(x)\right)$$

9.2.5. Simplified LS Estimation Methods

The recursive LS algorithm (9.18) and (9.19) requires the updates of $P(k)$ which involves matrix inversion, where $P(k)$ has a size of $n \times n$, and n r2 the number of parameters to be estimated. When n is large, this calculation can be time consuming.

Consider interpreting the cost function (9.6) as follows.

$$\hat{\theta}(k) = \arg\min_{\hat{\theta}} V_k(\hat{\theta}) = \arg\min_{\hat{\theta}}\frac{1}{2}\sum_{i=1}^{k}\left(y(i)-\phi(i)^T\hat{\theta}\right)^2 \tag{9.21}$$

$$\hat{\theta}(k+1) = \arg\min_{\hat{\theta}} V_{k+1}(\hat{\theta}) = \arg\min_{\hat{\theta}}\frac{1}{2}\sum_{i=1}^{k+1}\left(y(i)-\phi(i)^T\hat{\theta}\right)^2 \tag{9.22}$$

where

$$V_i(\hat{\theta}) \quad = \quad \frac{1}{2}\sum_{j=1}^{i}\left(y(j)-\phi(j)^T\hat{\theta}\right)^2$$

is the cost function formulated using the first i measurements $y(1),...,y(i)$.

Formulating (9.22) as a function of $V_k(\hat{\theta})$ gives

$$\hat{\theta}(k+1) \quad = \quad \arg\min_{\hat{\theta}}\left[V_k(\hat{\theta})+\frac{1}{2}\left(y(k+1)-\phi(k+1)^T\hat{\theta}\right)^2\right] \tag{9.23}$$

We already know that $\hat{\theta}(k)$ minimizes $V_k(\hat{\theta})$ from (9.21), and $V_k(\hat{\theta})$ happens to be the first term in (9.23). Therefore, (9.23) is essentially an optimization problem that concerns the following terms

$$\|\hat{\theta}-\hat{\theta}(k)\| \tag{9.24}$$

and

$$\|y(k+1)-\phi(k+1)^T\hat{\theta}\| \tag{9.25}$$

where $\|\cdot\|$ denotes the Euclidean norm of a vector, and the weights associated with the two terms in the optimization problem are determined by the covariance of the state estimate and measurement noise.

Consider a modification to the optimization problem in (9.23) as follows. Let (9.24) be the only term in the optimization, and (9.25) be a equality constraint, i.e.,

$$\min \qquad \frac{1}{2}\left(\hat{\theta}-\hat{\theta}(k)\right)^T\left(\hat{\theta}-\hat{\theta}(k)\right) \tag{9.26}$$

$$\text{s.t.} \qquad y(k+1)-\phi(k+1)^T\hat{\theta}=0 \tag{9.27}$$

Notice that this modified optimization problem given by (9.26) does not necessarily give the same result as the original problem in (9.23). This is because $\hat{\theta}(k+1)$ in (9.23) may not meet the equality constraint. Nevertheless, (9.26) is an approximation to $\hat{\theta}(k+1)$ in (9.23).

Optimization (9.26) can be solved using Lagrange multiplier method. Details are neglected here, and the result is given by

$$\hat{\theta}(k+1) \quad = \quad \hat{\theta}(k)+\frac{\phi(k+1)}{\phi(k+1)^T\phi(k+1)}\left(y(k+1)-\phi(k+1)^T\hat{\theta}(k)\right) \tag{9.28}$$

Parameters $0<\gamma<2$ and $\alpha>0$ are added to (9.28) as shown below. The motivations of adding γ and α are to make it possible to change the step length of

the parameter adjustment and to avoid a potential problem what would occur when $\phi(k+1) = 0$.

$$\hat{\theta}(k+1) \quad = \quad \hat{\theta}(k) + \frac{\gamma\phi(k+1)}{\alpha + \phi(k+1)^T\phi(k+1)} \Big(y(k+1) - \phi(k+1)^T\hat{\theta}(k)\Big) \tag{9.29}$$

which is known as the projection algorithm. The choice $0 < \gamma < 2$ guarantees the convergency of (9.29).

One may think the quality constraint (9.27) is too strong. When measurement error is considered, a modified version of (9.28) is given below.

$$\hat{\theta}(k+1) \quad = \quad \hat{\theta}(k) + \frac{\phi(k+1)}{\sum_{i=1}^{k+1} \phi(i)^T\phi(i)} \Big(y(k+1) - \phi(k+1)^T\hat{\theta}(k)\Big) \tag{9.30}$$

which is known as the stochastic approximation (SA) algorithm.

A further simplification to (9.29) and (9.30) is given below.

$$\hat{\theta}(k+1) \quad = \quad \hat{\theta}(k) + \gamma\phi(k+1) \Big(y(k+1) - \phi(k+1)^T\hat{\theta}(k)\Big)$$

which is known as the least mean square (LMS) algorithm.

9.2.6. Plant Models for LS Estimation

The LS estimation algorithm can be applied to the models introduced in this section. By assuming a model, experiments can be designed and empirical data can be collected which serves as the input to the LS estimation algorithm. The parameters of the model can then be decided.

The initial states of the plant is ignored for now. In practice, with the model and parameters known, state estimation is carried out to determine the state of the system.

Finite Impulse Response (FIR) Model

An LTI system can be uniquely characterized by its impulse response. A demonstrative figure is given in Fig. 9.6. For an asymptotically stable system, the transient will converge to zero exponentially fast, depending on the position of the poles in the transfer function.

Consider sampling the outputs of Fig. 9.6 in discrete time domain. The input at k, i.e., $u(k)$, would affect the consequent outputs $y(k+1), y(k+2), ...$ This sequence can go infinitely long in theory, but in practice we only consider the first finite number of outputs before y settles to zero. The length of the sequence is determined by the sampling period and the time constant of the system.

Due to the superposition of the LTI system, the above also indicates that output of the system at k, i.e., $y(k)$, is affected by the past input signals $u(k-1), u(k-2), ...,$

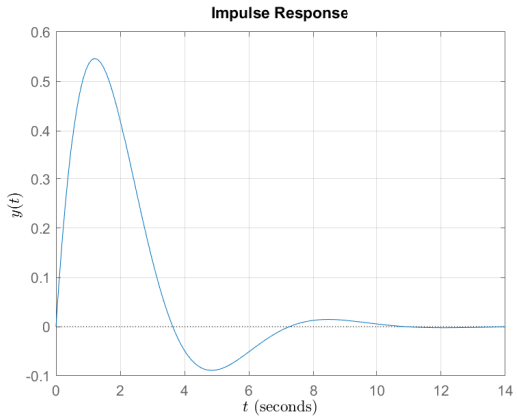


Figure 9.6.: LTI system impulse response demonstration.

as given by the following equation. Likewise, only finite number of inputs are included in the equation.

$$y(k) = b_1u(k-1) + b_2u(k-2) + \ldots + b_nu(k-n) + \epsilon(k) \tag{9.31}$$

Equation (9.31) is known as the FIR model of a system. It has n parameters to be estimated, and it fits well into the LS estimation schema described by (9.1).

General Transfer Function Model

Consider a more general LTI model described by the following input-output relations.

$$\begin{aligned} A(q)y(k) &= B(q)u(k) \\ A(q) &= q^n + a_1q^{n-1} + \ldots + a_n \\ B(q) &= b_1q^{m-1} + b_2q^{m-2} + \ldots + b_m \end{aligned}$$

where q is the forward shift operator $qy(k) = y(k+1)$. The goal of the parameter estimation is to estimate parameters $a_i, i = 1, ..., n$ and $b_i, i = 1, ..., m$.

Rewrite (9.32) as follows.

$$\begin{aligned} y(k) = & -a_1y(k-1) - a_2y(k-2) - \ldots - a_ny(k-n) \\ & +b_1u(k-1) + b_2u(k-2) + \ldots + b_mu(k-m) \end{aligned} \tag{9.32}$$

In (9.32), treat $y(k-1), ..., y(k-n)$ and $u(k-1), ..., u(k-m)$ as regressors and a LS estimation problem can be formulated.

Nonlinear Model

LS estimation can be applied to nonlinear models as long as the output to the parameters to be estimated are of linear relationship. An example is given as follows. Consider

$$y(k) + ay(k-1) \quad = \quad b_1 u(k-1) + b_2 \sin\left(u(k-2)\right)$$

This is a nonlinear model due to the sine function applied to $u(k-2)$. However, it has no impact on parameter estimation, as $y(k)$ is linear to a, b_1 and b_2. The nonlinear portion is wrapped into the regressor.

Stochastic Model

LS estimation is designed to handle independent and identically distributed Gaussian noise. This indicates that the noise should be white noise, i.e., $\epsilon(i)$ and $\epsilon(j)$, $i \neq j$ shall be independent. In addition, the noise shall not correlate with the regressor by any means.

In the case where the noise are not while, i.e., $\epsilon(i)$ and $\epsilon(j)$ correlates for some $i \neq j$, it is possible to get the parameters estimate as follows. Formulate the system dynamic model as

$$\begin{aligned} A(q)y(k) &= B(q)u(k) + C(q)\varepsilon(k) \\ y(k) &= -a_1 y(k-1) - \ldots - a_n y(k-n) \\ &\quad +b_1 u(k-1) + \ldots b_m u(k-m) \\ &\quad +c_1 \varepsilon(k-1) + \ldots + c_n \varepsilon(k-n) \end{aligned} \tag{9.33}$$

where $\varepsilon(k)$ are re-formulated i.i.d. noise and $C(q)$ is used to describe the correlation of the noise. The parameters in $C(q)$ will be estimated together with $A(q)$ and $B(q)$ to get a sense of the bandwidth of the original noise $\epsilon(k)$.

Notice that (9.33) is different from (9.32) because ε on the right side of (9.33) is unknown, while everything on the right side of (9.32) is known. To tackle this issue, measurement residual is used to approximate ε, i.e.,

$$\begin{aligned} e(k) &= y(k) - \psi(k-1)^T \hat{\theta}(k-1) \\ \psi(k-1) &= \left[\begin{array}{cccc} -y(k-1) & \ldots & -y(k-n) \\ u(k-1) & \ldots & u(k-m) \\ e(k-1) & \ldots & e(k-n) \end{array} \right]^T \\ \theta &= \left[\begin{array}{ccccccccc} a_1 & \ldots & a_n & b_1 & \ldots & b_m & c_1 & \ldots & c_n \end{array} \right]^T \end{aligned} \tag{9.34}$$

is used as a replacement of $\varepsilon(k)$ when formulating the parameter estimation. Notice that $\hat{\theta}(k-1)$ is the most recent parameter estimation update before receiving $y(k)$. This method is known as the extended least square estimation.

There is a similar alternative where the residual $e(k)$ is estimated using another regression model

$$\hat{C}(q)e(k) \quad = \quad \hat{A}(q)y(k) - \hat{B}(q)u(k)$$

instead of (9.34). This is known as the recursive maximum likelihood method.

9.2.7. Practical Issues in Parameter Estimation

The precision of the parameter estimate is largely affected by practical issues such as experimental conditions. In performing system identification automatically as in adaptive control, the precision can be crucial to the performance of the control system. This section studies the influence of the practical issues to the parameter estimate precision.

Excitation Condition in FIR Model

As introduced earlier, to carry out parameter estimation for an FIR model given by (9.31), a common way is to supply a series of control signal to the model, and use the output of the model to do LS estimation. The control signal needs to be designed strategically so that the excitation condition is satisfied.

Substitute (9.31) into (9.1) and (9.2) to get the excitation condition for the FIR model as follows.

Part IV.

System Identification

10. Parameter and State Estimation

11. Optimal State Estimation

Optimal and robust estimation methods are introduced in this chapter, many of which Kalman filter and its variations.

11.1. Weighted Least Squares Estimation

Consider the following system with only a linear measurement model

$$y = H\theta + v \tag{11.1}$$

where $y \in \mathbb{R}^{m\times 1}$, $\theta \in \mathbb{R}^{n\times 1}$ are the measurement vector and the parameter to be estimated, respectively. Matrix $H \in \mathbb{R}^{m\times n}$ is the measurement matrix, where $m \geq n$ and $H^T H$ is non-singular matrix with full rank n to guarantee instantaneous observability. Vector $v \in \mathbb{R}^{m\times 1}$ is zero-mean Gaussian-distributed measurement noise with covariance matrix R which is symmetric and positive-definite.

For simplicity of derivation, in the remaining of this section, we assume that each entry in v follows independent and identically distributed (i.i.d.) Gaussian distribution, making R a diagonal matrix. Similar conclusions can be extended to correlated measurement noise.

The maximum likelihood estimation (MLE) of (11.1) is given by

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\hat{\theta}} \prod_{i=1}^{m} \frac{1}{\sqrt{2\pi\sigma_i^2}} e^{-\frac{\left(y_i - H_i\hat{\theta}\right)^2}{2\sigma_i^2}} \tag{11.2}$$

where $\sigma_i^2 = R_{ii}$ is the variance of measurement noise v_i.

By taking the natural logarithm, equation (11.2) is equivalent to

$$\begin{aligned}\hat{\theta}_{\text{MLE}} &= \arg\min_{\hat{\theta}} \sum_{i=1}^{m} -\frac{\left(y_i - H_i\hat{\theta}\right)^2}{2\sigma_i^2} \\ &= \arg\min_{\hat{\theta}} J\end{aligned}$$

where J is the cost function given by

$$J = -\sum_{i=1}^{m} \frac{\left(y_i - H_i\hat{\theta}\right)^2}{2\sigma_i^2} \tag{11.3}$$

Cost function J in (11.3) is a convex function of θ and it has unique global minimum. From (11.3),

$$\frac{\partial J}{\partial \hat{\theta}} = \sum_{i=1}^{m} H_i^T \frac{1}{\sigma_i^2} \left(y_i - H_i \hat{\theta} \right) \tag{11.4}$$

$$= H^T R^{-1} \left(y - H\hat{\theta} \right) \tag{11.5}$$

Derivative with respect to a vector:

Equation (11.4) is a derivative of a scalar J with respect to (w.r.t.) a column vector $\hat{\theta} \in \mathbb{R}^{n \times 1}$. The result can be either a $1 \times n$ row vector (known as numerator layout) or a $n \times 1$ column vector (known as denominator layout). Both notations are correct so long as they are kept consistent.

Equations (11.4) and (11.5) use denominator layout. If nominator layout were used, (11.4) and (11.5) should have been

$$\frac{\partial J}{\partial \hat{\theta}} = \sum_{i=1}^{m} \frac{1}{\sigma_i^2} \left(y_i - H_i \hat{\theta} \right) H_i$$

$$= \left(y - H\hat{\theta} \right)^T R^{-1} H$$

In the next step, (11.5) will be equating to zero, hence making no difference beyond that point no matter which layout is used.

Equating $\dfrac{\partial J}{\partial \hat{\theta}} = 0$ in (11.5) gives

$$H^T R^{-1} y = H^T R^{-1} H \hat{\theta}_{\text{MLE}}$$

$$\hat{\theta}_{\text{MLE}} = \left(H^T R^{-1} H \right)^{-1} H^T R^{-1} y \tag{11.6}$$

where notice that $\left(H^T R^{-1} H \right)$ in (11.6) must be non-singular, which can be ensured if the system is instantaneously observable and R a positive-definite covariance matrix.

The state estimate (11.6) is non-biased provided that v is zero-mean. From (11.1) and (11.6), its estimation error is given by

$$\tilde{\theta}_{\text{MLE}} = \hat{\theta}_{\text{MLE}} - \theta$$

$$= \left(H^T R^{-1} H \right)^{-1} H^T R^{-1} y - \left(H^T R^{-1} H \right)^{-1} H^T R^{-1} H \theta$$

$$= \left(H^T R^{-1} H \right)^{-1} H^T R^{-1} v \tag{11.7}$$

From (11.7) and using $R = E[vv^T]$, the mean and covariance of WLS are

$$E\left[\tilde{\theta}_{\text{MLE}} \right] = 0$$

$$\text{Cov}\left[\tilde{\theta}_{\text{MLE}} \right] = E\left[\tilde{\theta}_{\text{MLE}} \tilde{\theta}_{\text{MLE}}^T \right]$$

$$= \left(H^T R^{-1} H \right)^{-1} \tag{11.8}$$

11.2. Kalman Filter

Consider the following system

$$x(k+1) = A(k)x(k) + G(k)w(k) \tag{11.9}$$
$$y(k) = C(k)x(k) + v(k) \tag{11.10}$$

where k is the time index, $x(k) \in \mathbb{R}^{n\times 1}$ the state vector, $A(k)$ the process matrix, $G(k)$ the process noise matrix, $w(k) \in \mathbb{R}^{l\times 1}$ the process noise, $y(k) \in \mathbb{R}^{m\times 1}$ the measurement, $C(k)$ the measurement matrix, and $v(k)$ the measurement noise. Noise $w(k)$ and $v(k)$ follow zero-mean Gaussian distribution with covariance $Q(k)$ and $R(k)$ respectively.

Notice that the control term $B(k)u(k)$ that could have been part of (11.9) is neglected for simplicity as this document only studies estimation and the control effect $B(k)u(k)$ is assumed known [XXX]. All the derived results in this section can be easily extended to a model with $B(k)u(k)$ considered in the process model.

The purpose of KF is to recursively estimate $\hat{x}(k)$ and its error covariance $P(k)$, given (11.9), (11.10), and priori knowledge of $\hat{x}(k-1)$ and $P(k-1)$.

KF can be derived using variety of methods. Rigorous derivations can be found at [XXX] where Bayes theorem is used in the derivation, or [XXX] where KF is formulated as a recursive MLE and derived using Thomas algorithm. In this document, an intuitive derivation is given.

The following lemma 1 is introduced and will be used in later derivations.

Lemma 1 *Let $\hat{\theta}^-$ be an estimation of θ, with estimation error covariance P^-. Let y be a new set of measurement of system in* (11.1)*. The MLE of θ, $\hat{\theta}_{\text{MLE}}$, and its estimation error covariance, P, can be calculated as follows.*

$$\begin{aligned} P &= \text{Cov}[\tilde{\theta}_{\text{MLE}}] \\ &= \left(P^{-^{-1}} + H^T R^{-1} H\right)^{-1} \end{aligned} \tag{11.11}$$
$$\hat{\theta}_{\text{MLE}} = \hat{\theta}^- + PH^T R^{-1}\left(y - H\hat{\theta}^-\right) \tag{11.12}$$

The proof is given below.

Let the size of θ be n. Take the priori knowledge $\hat{\theta}^-$ as a direct measurement of θ as follows.

$$\hat{\theta}^- = \theta + \varepsilon \tag{11.13}$$

with covariance of the measurement noise ε given by P^-. Obviously, the WLS to (11.13) *yields state estimate $\hat{\theta}^-$ and estimation error covariance P^-.*

Put (11.13) *together with the new measurements* (11.1) *in vector form as follows*

$$\begin{bmatrix} \hat{\theta}^- \\ y \end{bmatrix} = \begin{bmatrix} I \\ H \end{bmatrix}\theta + \begin{bmatrix} \varepsilon \\ v \end{bmatrix} \tag{11.14}$$

where I is the identity matrix of rank n. The augmented noise $\bar{V} = [\varepsilon\ v]^T$ has covariance

$$\mathrm{Cov}\left[\bar{V}\right] = \mathrm{diag}\left(\begin{array}{cc} P^- & R \end{array}\right)$$

and note that

$$\mathrm{Cov}^{-1}\left[\bar{V}\right] = \mathrm{diag}\left(\begin{array}{cc} P^{-^{-1}} & R^{-1} \end{array}\right)$$

due to the block diagonal structure of $\mathrm{Cov}\left[\bar{V}\right]$.

From (11.6) *and* (11.8)*, applying WLS on* (11.14) *gives*

$$\begin{aligned}
\hat{\theta}_{\mathrm{MLE}} &= \left(\left[\begin{array}{cc} I & H \end{array}\right]\left[\begin{array}{cc} P^{-^{-1}} & 0 \\ 0 & R^{-1} \end{array}\right]\left[\begin{array}{c} I \\ H \end{array}\right]\right)^{-1}\left[\begin{array}{cc} I & H \end{array}\right]\left[\begin{array}{cc} P^{-^{-1}} & 0 \\ 0 & R^{-1} \end{array}\right]\left[\begin{array}{c} \hat{\theta}^- \\ y \end{array}\right] \\
&= \left(P^{-^{-1}} + H^T R^{-1} H\right)^{-1} P^{-^{-1}}\hat{\theta}^- + \left(P^{-^{-1}} + H^T R^{-1} H\right)^{-1} H^T R^{-1} y \qquad (11.15) \\
\mathrm{Cov}\left[\tilde{\theta}_{\mathrm{MLE}}\right] &= \left(\left[\begin{array}{cc} I & H \end{array}\right]\left[\begin{array}{cc} P^{-^{-1}} & 0 \\ 0 & R^{-1} \end{array}\right]\left[\begin{array}{c} I \\ H \end{array}\right]\right)^{-1} \\
&= \left(P^{-^{-1}} + H^T R^{-1} H\right)^{-1} \qquad (11.16) \\
&\triangleq P \qquad (11.17)
\end{aligned}$$

From (11.17)*,*

$$P^{-1} = P^{-^{-1}} + H^T R^{-1} H \qquad (11.18)$$

Using (11.18)*,* (11.12) *can be re-written as follows.*

$$\begin{aligned}
\hat{\theta}_{\mathrm{MLE}} &= \hat{\theta}^- + P\left(P^{-^{-1}}\hat{\theta}^- + H^T R^{-1} y - P^{-1}\hat{\theta}^-\right) \\
&= \hat{\theta}^- + P\left(H^T R^{-1} y - H^T R^{-1} H\hat{\theta}^-\right) \\
&= \hat{\theta}^- + PH^T R^{-1}\left(y - H\hat{\theta}^-\right) \qquad (11.19)
\end{aligned}$$

Equations (11.17) *and* (11.19) *are* (11.11) *and* (11.12) *respectively.*

Denote $\hat{x}(k-1)$ to be an estimation of $x(k-1)$ with estimation error covariance $P(k-1)$. From (11.9), an estimate of $x(k)$ can be obtained as follows.

$$\hat{x}^-(k) = A(k-1)\hat{x}(k-1) \qquad (11.20)$$

and its covariance error

$$\begin{aligned}
\mathrm{Cov}\left[\tilde{x}^-(k)\right] &= \mathrm{Cov}\left[\hat{x}^-(k) - x(k)\right] \\
&= A(k-1)P(k-1)A(k-1)^T + G(k-1)Q(k-1)G(k-1)^T \\
&\triangleq P^-(k) \qquad (11.21)
\end{aligned}$$

Consider new measurement $y(k)$ in (11.10). Using lemma 1 with $\hat{x}^-(k)$, $P^-(k)$ corresponding with θ^- and P^-, and (11.10) corresponding with (11.1), the updated state estimate, $\hat{x}(k)$, and its corresponding state estimate error covariance, $P(k)$, can be calculated using (11.11) and (11.12) as follows.

$$\begin{aligned} \hat{x}(k) &= \hat{x}^-(k) + P(k)C(k)^T R(k)^{-1}\left(y(k) - C(k)\hat{x}^-(k)\right) && (11.22) \\ P(k) &= \left(P^-(k)^{-1} + C(k)^T R(k)^{-1} C(k)\right)^{-1} && (11.23) \end{aligned}$$

Equations (11.20), (11.21), (11.22) and (11.23) form the recursive KF.

In some literatures, $\hat{x}^-(k)$ in (11.20) is denoted by $\hat{x}(k|k-1)$ and $P^-(k)$ in (11.21) by $P(k|k-1)$. State estimate $\hat{x}(k)$ in (11.22) is denoted by $\hat{x}(k|k)$ and $P(k)$ in (11.23) by $P(k|k)$. The calculation of $\hat{x}^-(k)$ using (11.20) is known as the time update (it is a prediction of the state using only the process model, without any input from the measurement), while the calculation of $\hat{x}(k)$ on top of $\hat{x}^-(k)$ using (11.22), the measurement update. The prediction $\hat{x}(k|k-1)$ is known as the priori estimate, while $\hat{x}(k)$, the posteriori estimate.

Define Kalman gain as follows.

$$K(k) = P(k)C(k)^T R(k)^{-1} \quad (11.24)$$

With (11.24), (11.22) can be written as

$$\hat{x}(k) = \hat{x}^-(k) + K(k)\left(y(k) - C(k)\hat{x}^-(k)\right) \quad (11.25)$$

The trace of $P(k)$ in (11.23) can be used to evaluate the performance of KF. It can be proved that for system described by (11.9) and (11.10), under white Gaussian noise assumption, KF is the optimal estimator, meaning that for any other non-biased estimator, its state estimate error covariance matrix must have a trace that is not less than that of KF. The proof is not given in this document.

It is worth introducing Kalman gain a bit further, as this concept will be used in UKF as well in Section 11.3. In (11.25), Kalman gain plays as an artifact gain factor that bridges the measurement residual $(y(k) - C(k)\hat{x}^-(k))$ to the state estimate $\hat{x}(k)$, balancing the influence of the residual versus the priori estimate $\hat{x}^-(k)$.

Using (11.23), (11.25) can be re-written as follows.

$$\begin{aligned} K(k) &= P(k)C(k)^T R(k)^{-1} \\ &= P(k)\left[C(k)^T R(k)^{-1}\left(R(k) + C(k)P^-(k)C(k)^T\right)\right]\left(R(k) + C(k)P^-(k)C(k)^T\right)^{-1} \\ &= P(k)\left[C(k)^T + C(k)^T R(k)^{-1} C(k)P^-(k)C(k)^T\right]\left(R(k) + C(k)P^-(k)C(k)^T\right)^{-1} \\ &= P(k)\left[P^-(k)^{-1}P^-(k)C(k)^T + C(k)^T R(k)^{-1} C(k)P^-(k)C(k)^T\right]\left(R(k) + C(k)P^-(k)C(k)^T\right)^{-1} \\ &= P(k)\left[P^-(k)^{-1} + C(k)^T R(k)^{-1} C(k)\right]P^-(k)C(k)^T\left(R(k) + C(k)P^-(k)C(k)^T\right)^{-1} \\ &= P(k)P(k)^{-1}P^-(k)C(k)^T\left(R(k) + C(k)P^-(k)C(k)^T\right)^{-1} \\ &= P^-(k)C(k)^T\left(R(k) + C(k)P^-(k)C(k)^T\right)^{-1} && (11.26) \\ &\triangleq P_{xy}(k)P_{yy}(k)^{-1} \end{aligned}$$

where

$$P_{yy}(k) = R(k) + C(k)P^{-}(k)C(k)^{T} \quad (11.27)$$
$$P_{xy}(k) = P^{-}(k)C(k)^{T} \quad (11.28)$$

Equation (11.26), (11.27) and (11.28) give more insights to Kalman gain. From (11.26), Kalman gain is composed of of two parts, $P_{yy}(k)^{-1}$ and $P_{xy}(k)$, where $P_{yy}(k)$ is the covariance of measurement residual $(y(k) - C(k)\hat{x}^{-}(k))$ (proof is straight forward), and $P_{xy}(k)$ the cross-covariance matrix that bridges measurement residual and state. Later in the introduction of UKF in Section 11.3, the covariance matrices cannot be traced analytically, and sigma points are used to represent them empirically. Nevertheless, Kalman gain composed of residual covariance and cross-covariance matrix is defined and used in the posteriori estimation loop in a similar manner.

Using matrix inverse lemma on (11.23) gives

$$P(k) = P^{-}(k) - P^{-}(k)C(k)^{T}\left(C(k)P^{-}(k)C(k)^{T} + R(k)\right)^{-1}C(k)P^{-}(k) \quad (11.29)$$

From (11.26) and (11.27), (11.29) can be re-written as

$$P(k) = P^{-}(k) - K(k)P_{yy}(k)K(k)^{T} \quad (11.30)$$

The measurement update (11.22) and (11.23) are equivalent to (11.26), (11.25) and (11.30).

11.3. Unscented Kalman Filter

Though being efficient, KF has at least the following limitations.

1. Both process and measurement models (11.9) and (11.23) must be linear, and must be known as their parameters are used in the recursive calculation.
2. For optimal performance, process and measurement noise $w(k)$, $v(k)$ in (11.9) and (11.23) must follow Gaussian distribution with known scale parameters.

UKF was proposed to address item 1 in the above list. Notice that UKF still assumes Gaussian distributed process and measurement noise, and it is not robust to outliers.

Consider non-linear system

$$x(k+1) = f_k\left(x(k), w(k)\right) \quad (11.31)$$
$$y(k) = g_k\left(x(k), v(k)\right) \quad (11.32)$$

where the dimensions of $x(k)$, $y(k)$, $w(k)$ and $v(k)$ be given by n, m, l_w, l_v respectively. Both process noise $w(k)$ and measurement noise $v(k)$ follow Gaussian distribution $w(k) \sim \mathcal{N}\left(0, Q(k)\right)$ and $v(k) \sim \mathcal{N}\left(0, R(k)\right)$ with $Q(k)$, $R(k)$ the covariance matrices. The process model f_k and g_k can be time-varying, as long as their inputs and outputs dimensions remain constant.

Apparently, KF given by (11.20), (11.21), (11.22) and (11.23) cannot be implemented on (11.31) and (11.32) directly for an obvious reason: Gaussian distribution does not retain in a non-linear transformation where superposition theorem does not apply.

UKF uses unscented transform (UT) to trace the mean and covariance of a Gaussian (or Gaussian-like) distribution passing through a non-linear mapping. More about UT is introduced in Section 11.3.1.

11.3.1. Unscented Transform

Let $f(x)$ be a non-linear function. Let $p(x)$ be the probability density function (PDF) of X, either Gaussian or Gaussian-like, with mean and covariance given by μ and σ^2 respectively. For simplicity, scalar X is assumed in this section unless otherwise mentioned. Let $Y = f(X)$ be another random variable derived from X. The target is to calculate the mean and covariance of Y.

The following 4 approaches are introduced, and their pros and cons compared. UT is one of them and will be introduced in the last.

- Linearization
- Monte-Carlo based method
- Gauss-Hermite quadrature
- Unscented transform

Notice that the first 3 approaches try to compute the following integral

$$I = \int_{-\infty}^{\infty} f(x)p(x)dx \tag{11.33}$$

efficiently, where $f(x)$ is any general non-linear function. If that could be done, the mean and covariance of Y can be obtained consequently in a straight forward manner. Different from them, UT directly approximates the mean and covariance of Y.

Linearization

The first-order Taylor expansion of $f(x)$ about μ is given by

$$f(x) \approx f(\mu) + \frac{d}{dx}f(x)\bigg|_{x=\mu}(x-\mu) \tag{11.34}$$

where $f(\mu)$, $\frac{d}{dx}f(x)\Big|_{x=\mu}$ are known constants. With Gaussian distributed $X \sim \mathcal{N}(\mu, \sigma^2)$, the mean and variance of $Y \approx f(\mu) + \frac{d}{dx}f(x)\Big|_{x=\mu}(X-\mu)$ can be easily calculated. The result is accurate if $f(x)$ is linear or of first-order polynomial, and it introduces approximation otherwise.

The drawbacks of this approach include:

- It is only a first-order Taylor approximation, and may perform badly even if $f(x)$ is "a little bit" non-linear.
- It requires the calculation of $\frac{d}{dx}f(x)\Big|_{x=\mu}$. In practice, this often means that the analytical form of f must be known.

Monte-Carlo based method

Generate N samples of X, namely X_i, $i = 1, ..., N$. Calculate $f(X_i)$. Calculate the sampled mean and covariance of $f(X_i)$, and use them as the mean and variance of Y.

The result is accurate as $N \to \infty$ as a result of the law of large numbers and the central limit theorem, regardless of the nonlinearity of f. The obtained mean and variance converges with the increment of N. The speed of convergence can be quantified.

The drawbacks include:

- In practice, it is never accurate due to the limited size of N, even for linear f.
- The computational burden is high for an practically acceptable accuracy.

Gaussian-Hermite quadrature

Gaussian-Hermite quadrature assumes Gaussian distribution of X. Rewrite (11.33) with $p(x)$ replaced by Gaussian distribution PDF as follows.

$$E[Y] = \int_{-\infty}^{\infty} f(x)\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}dx \tag{11.35}$$

A two-step approach is used. In step 1, map random variable X with Z as follows.

$$x = \mu + \sqrt{2}\sigma z \Leftrightarrow z = \frac{x-\mu}{\sqrt{2}\sigma} \tag{11.36}$$

which makes $Z \sim \mathcal{N}(0, 0.5^2)$ a normalized random variable derived from X. The purpose of this transformation will be clearer in a later stage.

Substituting (11.36) into (11.35) gives

$$\begin{aligned} E[Y] &= \frac{1}{\sqrt{\pi}}\int_{-\infty}^{\infty} f\left(\mu + \sqrt{2}\sigma z\right)e^{-z^2}dz \\ &= \frac{1}{\sqrt{\pi}}\int_{-\infty}^{\infty} \bar{f}(z)e^{-z^2}dz \end{aligned} \tag{11.37}$$

where

$$\bar{f}(z) = f\left(\mu + \sqrt{2}\sigma z\right)$$

It turned out that $\int_{-\infty}^{\infty} \bar{f}(z)e^{-z^2}dz$ can be calculated very efficiently and accurately. This is the reason (11.36) is performed in the first place. The method to calculate $\int_{-\infty}^{\infty} \bar{f}(z)e^{-z^2}dz$ is known as Gauss-Hermite quadrature.

Detailed introduction of Gauss-Hermite quadrature can be found elsewhere. It is a very-well studied subject and not the focus of this document. A brief review is given as a side note.

A brief review of Gauss-Hermite quadrature:

We want to calculate (an approximation of) the following integral efficiently.

$$\int_{-\infty}^{\infty} f(x)\omega(x)dx \tag{11.38}$$

where $f(x)$ is an arbitrary non-linear function, $\omega(x)$ a fixed known weight function (so that it does not add unknowns to the system). Gauss quadrature assumes general weight functions. Commonly seen ones include $\omega(x) = 1$, $\omega(x) = \sqrt{1-x^2}$, $\omega(x) = e^{-x}$, and a few more. As far as this document concerns, in (11.37), $w(x) = e^{-x^2}$ is one of the commonly used weight functions. This special case of Gauss quadrature is known as the Gauss-Hermite quadrature. We will start by introducing general Gauss quadrature.

It is worth mentioning in the beginning that we can always use numerical integral to calculate (11.38), should there be sufficient computational resources. We can choose many samples of x_i in the given range, evaluate $f(x_i)$, then sum the product up. We can get accurate results to any degree should there be enough samples. However, this introduces huge computational burden.

Naturally, we want to set a benchmark for approximation accuracy, and use the least number of $f(x_i)$ evaluations to achieve that accuracy. In Gaussian quadrature problem formulation, the problem is stated as follows. For integration (11.38), we want to find such x_i and w_i so that

$$\int_{-\infty}^{\infty} f(x)\omega(x)dx \quad \approx \quad \sum w_i f(x_i) \tag{11.39}$$

is an approximation of the integral, and it is exact if $f(x)$ is a polynomial of order $(2N-1)$ or less. (After all, any non-linear $f(x)$ can be assumed to be a polynomial with very high order.) The problems are: how many x_i we need (the less the better), where to place them, and how to calculate w_i.

From the degree of freedom perspective, a polynomial of $(2N-1)$ order has $2N$ coefficients. By adding a evaluation point, we add 2 degrees of freedom to the right side of (11.39). Technically speaking, we need at least N evaluation point.

It is not recommended to randomly choose the locations of x_i, as one may realize later that the corresponding w_i is difficult to calculate, does not exist, not unique, or making the approximation numerically unstable. It is recommended to choose x_i according to the weight function $\omega(x)$. To be more precise, x_i should be the roots of the N-th order polynomial $\phi_N(x)$ that comes from the orthogonal

system $\{\phi_i(x)\}$ where

$$\int \phi_i(x)\phi_j(x)\omega(x)dx = 0, \qquad \forall i \neq j$$

In our case, $\omega = e^{-x^2}$ from (11.37), and the orthogonal system is known as the Physicists' Hermite polynomials. The i-th order Physicists' Hermite polynomial is often denoted by $H_i(x)$. As aforementioned, in our case, x_i should be chosen as the roots of $H_N(x)$.

Gaussian quadrature with weight function $\omega = e^{-x^2}$ is known as Gauss-Hermite quadrature. Gauss-Hermite quadrature states that

$$\int_{-\infty}^{\infty} f(x)e^{-x^2}dx \approx \sum_{1}^{N} w_i f(x_i) \tag{11.40}$$

where N is up to the user's choice, x_i, $i = 1, ..., N$ the roots to $H_N(x) = 0$, and w_i the weights given by

$$w_i = \frac{2^{N-1}N!\sqrt{\pi}}{N^2\left(H_{N-1}(x_i)\right)^2}$$

The explanations why polynomials from orthogonal family should be used and how w_i is calculated are out of the scope of this document.

The covariance can be calculated similarly.

Unscented Transform

GHQ is the product of mathematicians. It is precise, elegant and optimal, with minimum user intervention. The only thing a user can decide in GHQ is to choose N, which controls the accuracy of the approximation when calculating the mean and covariance. UT, on the other hand, is invented by engineers. It is not as elegant or rigorous. It is sub-optimal in many occasions. It has more parameters that the user can tune, some work better than the others and it is often case-by-case. But it works in practice, especially for high-dimension systems, whereas in many such cases GHQ does not.

The idea of UT can be summarized as follows. Strategically select a few sigma points of x_i. The number of sigma points depend on the dimension of the system. In the case of 1-D system where x is a scalar, select 3 sigma points, x_0, x_1 and x_2, where

$$\begin{aligned} x_0 &= \mu \\ x_1 &= x_0 + \delta \\ x_2 &= x_0 - \delta \end{aligned}$$

i.e., x_0 lies on the mean of its distribution and x_1, x_2 symmetrically spread to the right and left of the mean by δ, a tuned parameter. Notice that δ reflects the spread of the

sigma points, and it is a tune-able parameter. In practice, δ is mostly (not entirely) determined by the variance of X. More details will be given later.

Now that the sigma points locations are determined, we want to calculate their corresponding weights, w_0, w_1 and w_2, so that

$$\begin{aligned} w_1 &= w_2 \\ \mu &= \sum_{i=0}^{2} w_i x_i \qquad (11.41) \\ \sigma^2 &= \sum_{i=0}^{2} w_i \left(x_i - \mu\right)^2 \qquad (11.42) \end{aligned}$$

where (11.41) and (11.42) reflect the mean and variance constraints. The idea of UT is to find such sigma points and associated weights that their weighted sum can be used to reflect the original distribution's mean and variance. There are 3 weights and 3 equations, and hence the weights can be uniquely determined as follows.

$$\begin{aligned} w_0 &= 1 - \frac{\sigma^2}{\delta^2} \\ w_1 = w_2 &= \frac{\sigma^2}{2\delta^2} \end{aligned}$$

Since $x_0 = \mu$ is fixed, w_0 is not relevant in the calculation of (11.42). Noticing this, we can introduce another degree of freedom to the weights without violating the mean and variance constraints. Define mean weights w_i^m and covariance weights w_i^c as follows.

$$\begin{aligned} w_i^m &= w_i \\ w_0^c &= w_0 + (1 - \alpha^2 + \beta) \\ w_i^c &= w_i, i \geq 1 \end{aligned}$$

This adds additional degree of freedom to w_0^c, where α and β are tune-able parameters with typical values 0.001 and 2 respectively. Good choices of α and β help with mapping the skewness and kurtosis of the PDF for Gaussian-like distributions. Details are not covered here.

To conclude, and to generalize the problem to L-dimension Gaussian-like distributed $x \in \mathbb{R}^{L\times 1}$ with mean vector $\mu \in \mathbb{R}^{L\times 1}$ covariance matrix $P_x \in \mathbb{R}^{L\times L}$, UT approximates the mean and covariance of $Y = f(X)$ as follows. It selects $(2L + 1)$ sigma points at

$$x_i = \begin{cases} \mu & i = 0 \\ \mu + \delta_i & i = 1, ..., L \\ \mu - \delta_{i-N} & i = L + 1, ..., 2L \end{cases} \qquad (11.43)$$

where δ_i is a tune-able parameter (vector) mostly determined by P_x. In practice, we use

$$\delta_i = \left(\sqrt{(L + \lambda)P_x}\right)_i$$

where $\sqrt{\cdot}$ is the matrix square root, usually Cholesky decomposition, and $(\cdot)_i$ the i-th column of a matrix. Parameter λ is a tune-able parameter calculated by $\lambda = \alpha^2(L+\kappa) - L$. Typical values are $\alpha = 0.001$ and $\kappa = 0$. Notice that λ can be negative.

The mean weights and covariance weights are denoted by w_i^m and w_i^c, $i = 0, ..., 2L$. They are calculated as follows.

$$w_0^m = \frac{\lambda}{L+\lambda} \tag{11.44}$$

$$w_0^c = \frac{\lambda}{L+\lambda} + \left(1 - \alpha^2 + \beta\right) \tag{11.45}$$

$$w_i^m = w_i^c = \frac{1}{2(L+\lambda)} \tag{11.46}$$

The typical values of UT parameters:

If the typical values of parameters in UT sounds strange to you, you are not along. For example, in $\lambda = \alpha^2(L+\kappa) - L$, by using typical values $\alpha = 0.001$ and $\kappa = 0$, $\lambda \approx -L$. This makes the sigma points spread δ_i very small, and all sigma points almost collapse to μ. Indeed, several studies have pointed this problem out and they tried to propose better guidance on how to select all the tune-able the parameters. The guidance works better in some cases while worse in others.

This document does not bother the details on how to select the tune-able parameters at what conditions. Throughout the document, typical values are used. And they work.

With the above sigma points x_i and weights w_i^m, w_i^c, (notice that they purely depend on the distribution of X and has nothing to do with $f(x)$), the mean and covariance of $Y = f(X)$ are approximated as follows.

$$E[Y] \approx \sum_{i=0}^{2L} w_i^m f(x_i)$$

$$\mathrm{Cov}[Y] \approx \sum_{i=0}^{2L} w_i^c \left[f(x_i) - E[Y]\right] \left[f(x_i) - E[Y]\right]^T$$

11.3.2. Unscented Kalman Filter

UKF is KF with mean (state estimate) and covariance (state estimate error covariance) non-linear mapping replaced by UT. To capture the non-linear process and measurement model, state augmentation is used. Details are given below.

Consider (11.31) and (11.32). Assume that at $k-1$, the state estimate is given by $\hat{x}(k-1)$ with state estimate error covariance $P(k-1)$. The target is to calculate state estimate $\hat{x}(k)$ using the process model, the measurement model, and the newly measured $y(k)$.

Let augmented $\bar{X}$ be

$$\bar{X}(k-1) \quad = \quad \begin{bmatrix} \hat{x}(k-1)^T & 0_{l_m} & 0_{l_v} \end{bmatrix}^T \tag{11.47}$$

an $(n + l_m + l_v) \times 1$ column vector. Let augmented error covariance matrix be

$$\bar{P}(k-1) \quad = \quad \begin{bmatrix} p(k-1) & 0 & 0 \\ 0 & Q(k-1) & 0 \\ 0 & 0 & R(k) \end{bmatrix} \tag{11.48}$$

Generate sigma points of the above augmented states using (11.43). The number of sigma points is $(2L+1)$ where $L = n+l_m+l_v$. Each sigma point is an $(n+l_m+l_v)\times 1$ column vector. Form them together into an $(n + l_m + l_v) \times (2L + 1)$ matrix denoted by $\mathcal{X}$. This matrix serves as a pool of sigma points.

$$\mathcal{X} \quad = \quad \begin{bmatrix} \dots & \mathcal{X}_i & \dots \end{bmatrix}_{(n+l_m+l_v)\times(2L+1)} \tag{11.49}$$

Split the matrix by rows as follows.

$$\mathcal{X} \quad = \quad \begin{bmatrix} \mathcal{X}^x_{n\times(2L+1)} \\ \mathcal{X}^w_{l_w\times(2L+1)} \\ \mathcal{X}^v_{l_v\times(2L+1)} \end{bmatrix} \tag{11.50}$$

where the dimensions of $\mathcal{X}^x$, $\mathcal{X}^w$ and $\mathcal{X}^v$ are $n \times (2L + 1)$, $l_w \times (2L + 1)$ and $l_v \times (2L+1)$ respectively. They correspond with the state vector, the process noise and the measurement noise.

Calculate the mean weights w_i^m and covariance weights w_i^c associated with the sigma points using (11.44), (11.45) and (11.46).

In KF, time update is given by (11.20) and (11.21). The corresponding UKF time update is given below.

$$\hat{x}^-(k) \quad = \quad \sum_{i=0}^{2L-1} w_i^m f_{k-1}\left(\mathcal{X}_i^x, \mathcal{X}_i^w\right) \tag{11.51}$$

and its covariance error

$$P^-(k) \quad = \quad \sum_{i=0}^{2L-1} w_i^c \left[\mathcal{X}_i^x - \hat{x}^-(k)\right]\left[\mathcal{X}_i^x - \hat{x}^-(k)\right]^T \tag{11.52}$$

In KF, measurement update is given by (11.26), (11.25) and (11.30). The corresponding UKF measurement update is given below. Firstly, consider the calculation of $P_{yy}(k)$ and $P_{xy}(k)$ given by (11.27) and (11.28). Instead of analytical calculation,

they are approximated using sigma points as follows.

$$\hat{y}(k) = \sum_{i=0}^{2L-1} w_i^m g_k\left(f_{k-1}\left(\mathcal{X}_i^x, \mathcal{X}_i^w\right), \mathcal{X}_i^v\right) \tag{11.53}$$

$$P_{yy}(k) = \sum_{i=0}^{2L-1} w_i^c \left[g_k\left(f_{k-1}\left(\mathcal{X}_i^x, \mathcal{X}_i^w\right), \mathcal{X}_i^v\right) - \hat{y}(k)\right] \times \ldots \left[g_k\left(f_{k-1}\left(\mathcal{X}_i^x, \mathcal{X}_i^w\right), \mathcal{X}_i^v\right) - \hat{y}(k)\right]^T \tag{11.54}$$

$$P_{xy}(k) = \sum_{i=0}^{2L-1} w_i^c \left[\mathcal{X}_i^x - \hat{x}^-(k)\right] \times \ldots \left[g_k\left(f_{k-1}\left(\mathcal{X}_i^x, \mathcal{X}_i^w\right), \mathcal{X}_i^v\right) - \hat{y}(k)\right]^T \tag{11.55}$$

Substituting (11.54) and (11.55) into (11.26) gives Kalman gain in UKF

$$K(k) = P_{xy}(k) P_{yy}(k)^{-1} \tag{11.56}$$

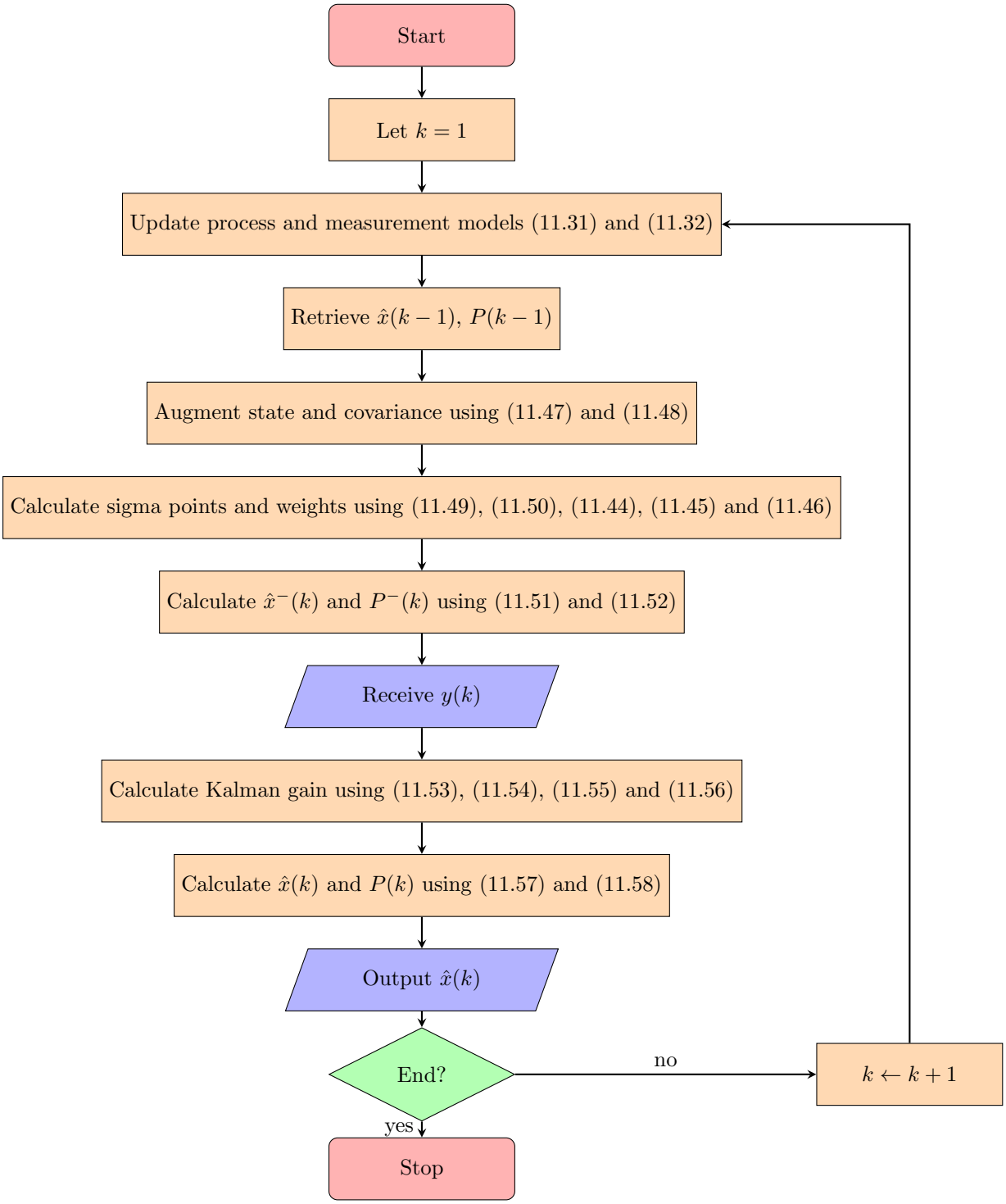
From (11.25) and (11.30), the state estimate and the state estimate error covariance can then be updated by

$$\hat{x}(k) = \hat{x}^-(k) + K(k)\left(y(k) - \hat{y}(k)\right) \tag{11.57}$$

$$P(k) = P^-(k) - K(k) P_{yy}(k) K(k)^T \tag{11.58}$$

where $\hat{y}(k)$ and P_{yy} are from (11.53) and (11.54) respectively. Equations (11.56), (11.57) and (11.58) form the measurement update of UKF, and they are corresponding with (11.26), (11.25) and (11.30).

A flowchart for UKF is given below. Notice that $\hat{x}(0)$, $P(0)$ in the flowchart represent the initial guess of the state and the initial guess error covariance. They are the priori knowledge of the state without any measurement.


Start
Let $k = 1$
Update process and measurement models (11.31) and (11.32)
Retrieve $\hat{x}(k-1)$, $P(k-1)$
Augment state and covariance using (11.47) and (11.48)
Calculate sigma points and weights using (11.49), (11.50), (11.44), (11.45) and (11.46)
Calculate $\hat{x}^-(k)$ and $P^-(k)$ using (11.51) and (11.52)
Receive $y(k)$
Calculate Kalman gain using (11.53), (11.54), (11.55) and (11.56)
Calculate $\hat{x}(k)$ and $P(k)$ using (11.57) and (11.58)
Output $\hat{x}(k)$
End?
no
$k \leftarrow k + 1$
yes
Stop

12. Robust State Estimation

13. Adaptive State Estimation

Part V.

Expanded Research Areas

14. Fuzzy Control System

Fuzzy control system provides a way of describing system attributes using vague languages and determining control based on rules of an expert system. With recent advent in large language model, people have seen potentials on fuzzy control system.

14.1. Introduction

Fuzzy logic provide a way of quantitatively interpreting and calculating attributes or reasoning described by vague languages. In contrast to classical control methods where variables are numbered precisely and control laws described clearly by mathematical equations, a fuzzy controller uses objective terms such as “good”, “bad”, “high”, “low”, “very”, “a little” to describe variables, and use rules “if something then something” to determine the control signals. This makes a fuzzy controller naturally good at realizing human knowledge base into practice.

Different from ANN which is highly data driven and requires a lot of samples and computations to train a network, a fuzzy controller is an expert-based system that requires only the membership functions used for fuzzification of variables and a set of rules for inference. ANN may reveal the building bricks of a human brain, while fuzzy control system reflects how a human interprets the knowledge that he learns from past experience and uses in his daily life. This feature of a fuzzy control system is a bless and a curse at the same time. Fuzzy control system does not rely on training, thus is handy when there are few training samples. On the other hand, this also means that it cannot benefit from the accumulation of the samples and it is lack of evolving capability comparing with ANN and other evolutionary algorithms.

14.2. Fuzzy Set and Fuzzy Relation

Fuzzy set is not a set from mathematical perspective, but a projection derived from a set. Fuzzy set and fuzzy relation are the fundamentals of fuzzy logics. Details are given in the remaining of this section.

14.2.1. Fuzzy Set

The fundamentals of fuzzy logics and fuzzy control systems are built on fuzzy set and membership function. Fuzzy set is not a set from mathematical perspective, but a projection from a set to a real number between $[0, 1]$. The function used in the projection is called the member function of the fuzzy set.

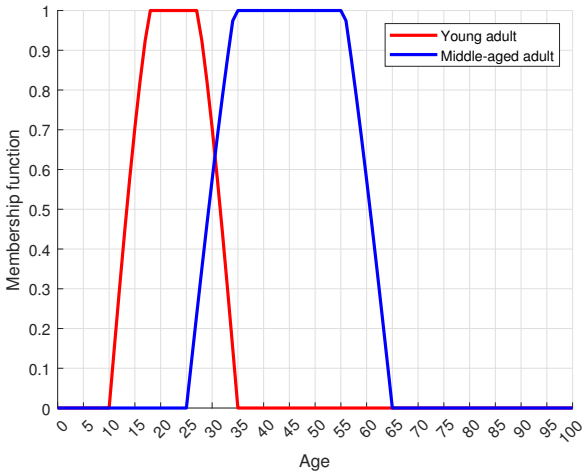


Figure 14.1.: Membership function of fuzzy sets "young adult" and "middle-aged adult".

Consider set U with finite or infinite cardinality. Define a projection $\mu_{\underset{\sim}{A}}$ from $x \in U$ to $[0, 1]$ as follows.

$$\mu_{\underset{\sim}{A}} \quad : \quad x \in U \rightarrow [0, 1]$$

This projection, together with its domain (also known as the universe of a fuzzy set) defines a fuzzy set. The projection $\mu_{\underset{\sim}{A}}$ is called the membership function of the fuzzy set $\underset{\sim}{A}$. The value of $\mu_{\underset{\sim}{A}}(x)$ for $x \in U$ describes in what extent x belongs to $\underset{\sim}{A}$.

It is clear from the definition the difference between a set and a fuzzy set. For a set, an element either belongs to the set or does not belong to the set, which can be described by

$$\mu_A(x) = \begin{cases} 1 & x \in A \\ 0 & x \notin A \end{cases}$$

where μ_A is a binary function. In contrast, fuzzy set defines a continuous functions $\mu_{\underset{\sim}{A}}$ which can take any value between $[0, 1]$. From this stand, a fuzzy set is an extension to a set.

An example of a fuzzy set is given below. Let $U \geq 0$ be the age of a person. Define 2 fuzzy sets, "young adult" as $\underset{\sim}{A}$ and "middle-aged adult" as $\underset{\sim}{B}$, with membership function defined in Fig. 14.1.

From Fig. 14.1, we can see how a person is categorized according to the age. For example, a 5-year-old child is by no means a young adult or an adult, as $\mu_A = \mu_B(5) = 0$ from the figure. A 27-year-old can be considered "quite" a young adult, with $\mu_A = 0.8$, and "somehow" adult, with $\mu_A = 0.3$.

The maximum value of a membership function is defined as the height of the fuzzy set. In many occasions the height is 1, but it is not always the case. An element whose associated membership function equals to half of the height is called the crossover point.

It is possible to set up a threshold for the membership function, and derive a set from a fuzzy set as follows.

$$\underset{\sim}{A}_{\alpha} = \left\{ x \middle| \mu_{\underset{\sim}{A}}(e) \geq \alpha \right\}$$

Notice that $\underset{\sim}{A}_{\alpha}$ is a set (not a fuzzy set) known as the cut set of $\underset{\sim}{A}$. A fuzzy set can have infinite number of cut sets by choosing different α. A fuzzy set can be derived from all its cut sets as follows.

$$\underset{\sim}{A} = \bigcup_{\alpha \in [0,1]} \alpha A_{\alpha}$$

where αA_{α} is a fuzzy set defined on A_{α} with, universe A_{α} and membership function value α. The $\cup$ calculates the union of fuzzy sets, more of which is introduced later.

Define the support and core of a fuzzy set as follows, respectively.

$$\begin{aligned} \text{supp}(\underset{\sim}{A}) &= \left\{ x \middle| \mu_{\underset{\sim}{A}}(x) > 0 \right\} \\ C(\underset{\sim}{A}) &= \left\{ x \middle| \mu_{\underset{\sim}{A}}(x) = 1 \right\} \end{aligned}$$

A fuzzy set $\underset{\sim}{A}$ can be represented by its base set A together with associated membership functions $\mu_{\underset{\sim}{A}}$. Alternatively, use the following

$$\begin{aligned} \underset{\sim}{A} &= \sum_i \frac{\mu_{\underset{\sim}{A}}(x_i)}{x_i} \\ \underset{\sim}{A} &= \int_E \frac{\mu_{\underset{\sim}{A}}(x)}{x} \end{aligned}$$

to represent a fuzzy set with discrete and continuous universe, respectively. Notice that in the above representation, "/", "$\sum$", and "$\int$" are not division, summation and integration, but just conventional notations used in the fuzzy set theory.

14.2.2. Fuzzy Set Operations

Operations such as union and intersection are defined on fuzzy sets. They are summarized in Table 14.1.

Notice that Table 14.1 gives only one out of many ways to define these operations. There are other ways to define them. For example, for complement, there is Sugeno's complement and Yager's complement as follows. The ones given in Table 14.1 is the most commonly used definition.

$$\begin{aligned} N_s(e) &= \frac{1-e}{1+\lambda e} \\ N_w(e) &= (1-a^w)^{\frac{1}{w}} \end{aligned}$$

Operation	Symbol	Membership Function
Union	$\mathcal{A} \cup \mathcal{B}$	$\mu_{\mathcal{A}\cup\mathcal{B}}(e) = \max(\mu_{\mathcal{A}}(e), \mu_{\mathcal{B}}(e))$
Intersection	$\mathcal{A} \cap \mathcal{B}$	$\mu_{\mathcal{A}\cap\mathcal{B}}(e) = \min(\mu_{\mathcal{A}}(e), \mu_{\mathcal{B}}(e))$
Complement	$\mathcal{A}'$	$\mu_{\mathcal{A}'}(e) = 1 - \mu_{\mathcal{A}}(e)$

Table 14.1.: Commonly used fuzzy set operations.

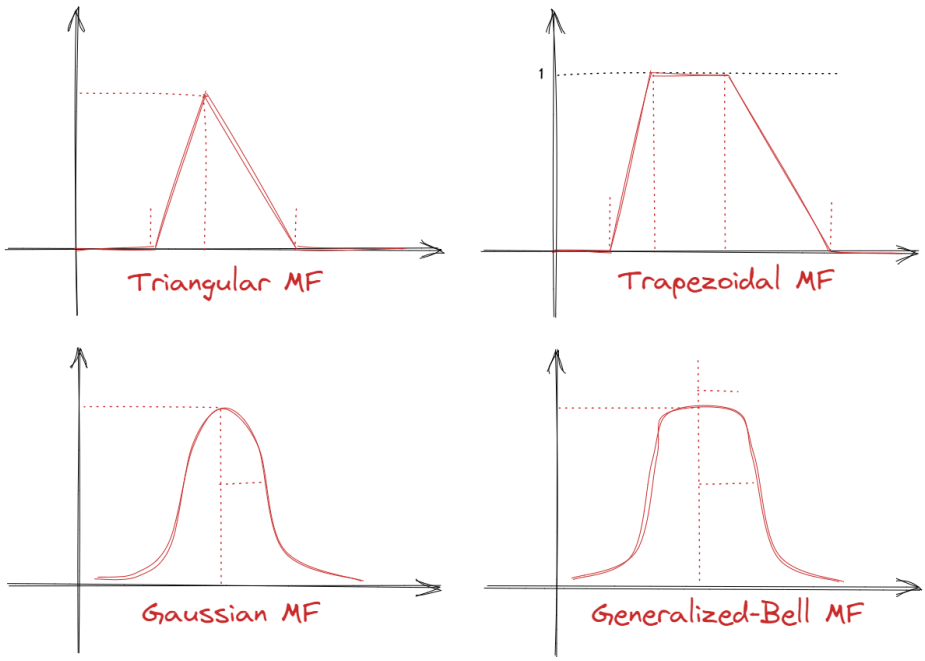


Figure 14.2.: Commonly used membership functions.

More operations can be found in [2].

14.2.3. Commonly Used Membership Functions

Membership functions can be chosen flexibly as long as it is bounded by $[0, 1]$ for each element. There are some commonly used ones that has been proved useful in general problems as shown in Fig. 14.2. They, together with their combinations, are widely used in fuzzy control system design.

MATLAB provides built-in functions to generate the functions in 14.2, such as `trimf` for triangular membership function, `trapmf` for trapezoidal function, `gaussmf` for Gaussian, and `gbellmf` for generalized-bell function, respectively. There are other membership functions built-in to MATLAB as well.

14.2.4. Fuzzy Relation

In set theory, a relation of two elements is defined using sets. For example, let $a, b \in \mathbb{R}$. Let a customized relation "$\oplus$" be defined on $\mathbb{R} \times \mathbb{R} = \mathbb{R}^2$. We can say $a \oplus b$ if and only if there is a set defined on $\mathbb{R}^2$ associated with "$\oplus$" relation, and (a, b) is in that set. Here "$\times$" denotes the Cartesian product.

Fuzzy relation is defined likewise, except that the set is replaced with fuzzy set as follows. Let $u \in U$, $v \in V$ be two variables with the same or different domains, and $D = U \times V$. The fuzzy relation of "$\oplus$" of two elements u and v is defined by the following fuzzy set

$$\mu_{\underset{\sim}{D}}(u, v) \quad \in \quad [0, 1]$$

associated with "$\oplus$".

It is possible to use a matrix to record the fuzzy relations, in which case the matrix is called a fuzzy matrix as it is made up of membership functions. Other choice include fuzzy graph, which takes advantage of graph theory to represent the fuzzy relation of different nodes in a universe.

The relation can be derived from chains. For example, consider relation P defined one $X \times Y$, and Q on $Y \times Z$. A relation on X, Z can be derived. One way (this is not the only way) is to use

$$\mu_{X \times Z}(x, z) \quad = \quad \sup\left[\min\left(\mu_{X \times Y}(x, y), \mu_{Y \times Z}(y, z)\right)\right]$$

14.3. Fuzzy Control System

Fuzzy control system is a practice of utilizing fuzzy logics in engineering. It mainly includes fuzzification of the physical measurement signals to fuzzy variables, fuzzy inference, and defuzzification of the fuzzy variables to physical control signals. Membership functions are used in the fuzzification and defuzzification of signals, and rules are used in the fuzzy inference.

A simple schema of a fuzzy control system is given in Fig. 14.3. Details are covered in the remaining of this section.

14.3.1. Fuzzification

xxx

14.3.2. Fuzzy Interface

xxx

14.3.3. Fuzzy Controller Design

xxx

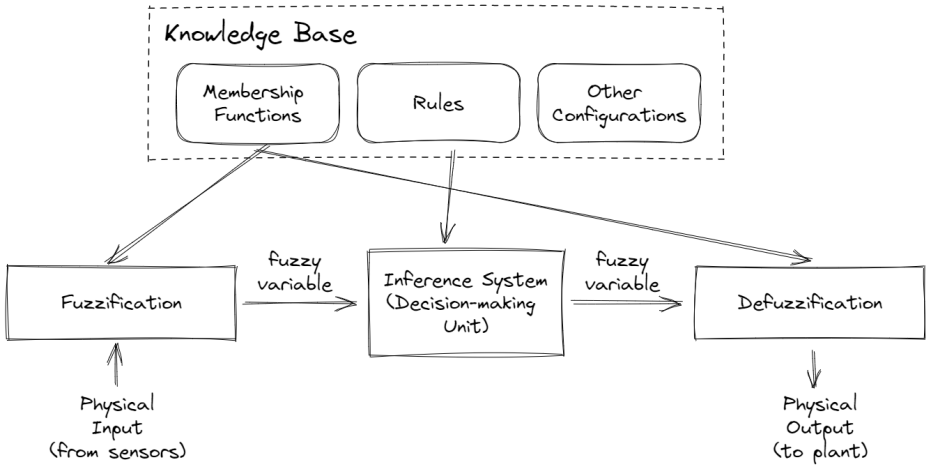


Figure 14.3.: A simple schema for a fuzzy control system.

14.3.4. Fuzzy Controller Performance Analysis

xxx

14.4. Fuzzy Modeling and Fuzzy System Identification

xxx

14.5. Fuzzy System Auto-tuning

The performance of a fuzzy control system is mainly determined by the choice of membership functions, inference rules and defuzzification methods. In the case where there is no such an expert that can carefully design everything, a data-driven approach can be implemented that automatically tunes the parameters setup in a fuzzy control system for the optimal performance.

Notice that the auto-tuning does not change the fact that a fuzzy control system is a rule-based expert system. Therefore, the advantage of fuzzy system being intuitive to interpret persists.

Nowadays, the most popular data-driven approach for data analysis is artificial neural network. It is possible to use ANN to find the optimal parameters of a fuzzy system, hence, fuzzy neural networks.

14.6. Fuzzy Control System Design Using MATLAB

MATLAB fuzzy logic toolbox provides tools to define membership functions and fuzzy inference systems. They are introduced in this section via examples.

14.6.1. Example: Vibration Detection

Consider the example given below. Design and implement the system in MATLAB as follows.

Vibration Detection from Measurement Signal

Oscillation has been detected from a continuously sampled signal. The oscillation might be caused by system process dynamics, system vibration, or measurement noise. The oscillation frequency and amplitude together help to reveal the cause of the oscillation, specifically the likelihood that the oscillation is caused by system vibration.

The input to the system is the oscillating measurement signal sampled at $1Hz$. An FFT is applied to the signal to get the oscillation frequency and amplitude, which lay in the range of $[0, 0.5](Hz)$ and $[0, 10]$, respectively. The oscillation frequency and amplitude is then sent to the fuzzy controller.

In the fuzzy controller, the two signals are fuzzified to form fuzzy variables. The fuzzy variables are passed to the fuzzy inference system, where rules are defined to determine the likelihood that the oscillation is caused by vibration, which is also a fuzzy variable. Finally, the likelihood fuzzy variable is defuzzified to obtain the vibration likelihood, which should be a variable in $[0, 1]$.

Membership Functions for Input and Outputs

The first step in solving this problem is to understand how an human expert would interpret oscillation frequency and amplitude, and derive the likelihood from them. Take oscillation frequency as an example. It is agreed on that a frequency that is too low or too high may indicate something other than vibration. For example, frequency lower than $0.05Hz$ will be categorized as "low". Similarly, frequency higher than $0.4Hz$ is probably too "high" for vibration. Frequency around $0.2Hz$ is the "fine" frequency, which increases the likelihood of vibration.

Imagine asking many experts on how they categorize each frequency $0.01Hz, 0.02Hz, ..., 0.5Hz$ into one of the 3 categories "low", "fine" or "high". Consider using the interview results to form the membership for the 3 categories as given in Fig. 14.4. The curves in Fig. 14.4 is obtained by fitting the "percentage of agreement" using trapezoidal functions. Figure 14.4 can be interpreted as in what extend one would agree that the specified frequency shall belong to a category.

The same applies to the oscillation amplitude, where 2 categories are defined, namely "low" and "fine", as shown by Fig. 14.5. Instead of using trapezoidal functions, generalized bell functions are used to fit the interview results.

With the above defined membership functions, each frequency and amplitude can be translated into a fuzzy variable using fuzzification. For example, frequency $0.12Hz$

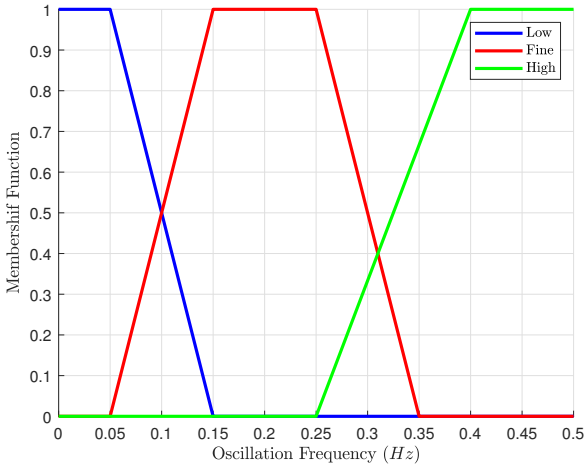


Figure 14.4.: Membership functions for 3 fuzzy sets “low”, “fine” and “high” defined on input signal oscillation frequency.

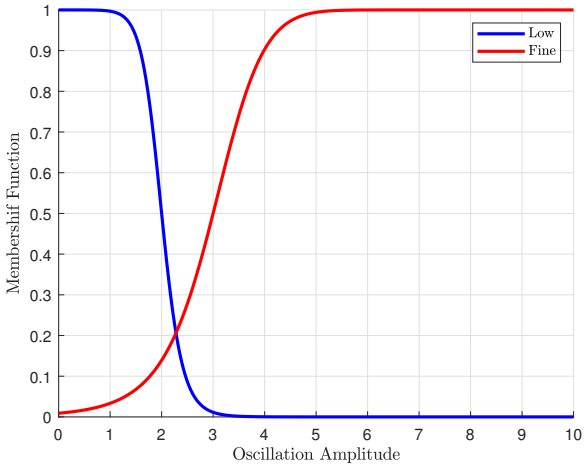


Figure 14.5.: Membership functions for 2 fuzzy sets “low” and “fine” defined on input signal oscillation amplitude.

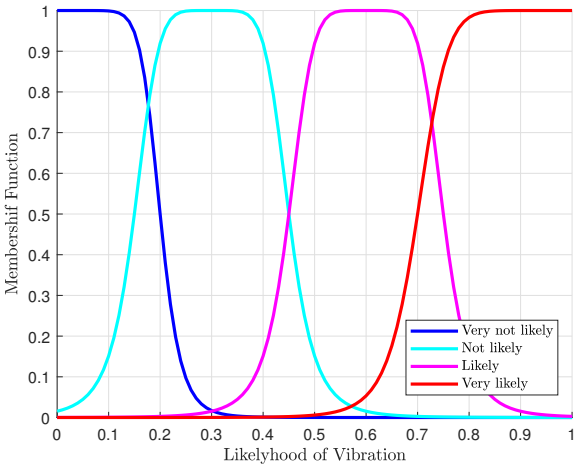


Figure 14.6.: Membership functions for 4 fuzzy sets "very unlikely", "unlikely", "likely" and "very likely", defined on input signal oscillation amplitude.

is $[0.3, 0.7, 0]$ where the three values in the vector represents the membership functions values of the given frequency mapping to each fuzzy set.

The output of the system is the likelihood of vibration of the system, scaled by a parameter in $[0, 1]$. Define 4 fuzzy sets, namely "very likely", "likely", "unlikely", "very unlikely", as shown in Fig. 14.6.

With the above defined membership functions, each likelihood fuzzy variable can be translated into the likelihood of the occurrence of vibration via defuzzification. For example, likelihood fuzzy variable $[0, 0.1, 0.3, 0.9]$ means that the the likelihood is not at all "very unlikely", a tiny little bit "unlikely", somewhat "likely", and quite much "very likely". Intuitively, this should translate into a high likelihood around $0.7 \sim 0.9$ using Fig. 14.6.

There are different ways of translating the output fuzzy variables into a physical signal, such as "centroid", "bisector", "middle of maximum", etc. More details can be found in MATLAB help document under "defuzzification methods". For example, $[0, 0.1, 0.3, 0.9]$ would translate into 0.73 using membership function given in Fig. 14.6 using centroid method, and 0.79 if using bisector method instead.

Inference

Fuzzy inference system is rule-based. Examples of the rules may include the following. Notice that in this example, all combinations of oscillation frequency and amplitude are listed. This is not always necessary in practice. Some of the rules may seem conflict with each other. This does not matter as the fuzzy controller will balance the weight of each output fuzzy set.

- If oscillation frequency is fine and oscillation amplitude is fine, likelihood of

vibration is very likely.

- If oscillation frequency is fine and oscillation amplitude is low, likelihood of vibration is likely.
- If oscillation frequency is high and oscillation amplitude is fine, likelihood of vibration is likely.
- If oscillation frequency is high and oscillation amplitude is low, likelihood of vibration is unlikely.
- If oscillation frequency is low and oscillation amplitude is fine, likelihood of vibration is likely.
- If oscillation frequency is low and oscillation amplitude is low, likelihood of vibration is very unlikely.
- If oscillation amplitude is fine, likelihood of vibration is likely.

It is worth mentioning that "AND", "OR", "NOT" can be used in the propositions, as shown above, for example "frequency is fine AND amplitude is fine". The membership function of "frequency is fine AND amplitude is fine" can be derived from the two individual membership functions following rules defined in Table 14.1 or its alternatives.

The output fuzzy sets are the "pools" that collect scores. The input is checked against each rule, which add scores to its associated pools. The scores form the value output fuzzy variable.

Realization

With the above, it is possible to program the fizzy control system from scratch. Alternatively, fuzzy inference systems tools provided by MATLAB can also be used to quickly deploy a fuzzy control system. Both methods are investigated as follows.

Consider programming from scratch. The following MATLAB program can be used to detect vibration. Notice that for simplicity, the membership functions and rules are hard-coded into the function.

```
fs = 1;                              % Sampling frequency
T = 1/fs;                            % Sampling period
L = 600;                             % Number of samples
t = (0:L-1)*T;                       % Time vector
measured_torque = 5*sin(2*pi*0.2*t);
measured_torque = measured_torque + 2*sin(2*pi*0.02*t);
measured_torque = measured_torque + normrnd(0, 1, [1, length(measured_torque)])
    ;
detect_vibration(fs, measured_torque)

function vibration_likelihood = detect_vibration(fs, measured_torque)
       % fft analysis to get the oscillation frequency and amplitude
       T = 1/fs;
```

```
L = length(measured_torque);
t = (0:L-1)*T;
figure
plot(t, measured_torque, 'b-')
grid on
xlabel("Time", "Interpreter", "latex")
ylabel("Torque", "Interpreter", "latex")
measured_torque_fft = fft(measured_torque);
P2 = abs(measured_torque_fft/L);
P1 = P2(1:L/2+1);
P1(2:end-1) = 2*P1(2:end-1);
f = fs*(0:(L/2))/L;
figure
plot(f, P1)
grid on
title("Single-Sided Amplitude Spectrum of Measured Torque")
xlabel("$f$ ($Hz$)", "Interpreter", "latex")
ylabel("$|P_1(f)|$", "Interpreter", "latex")
maj_f = f(P1 == max(P1));
maj_P1 = max(P1);
% fuzzification
x_freq = 0:0.01:0.5;
mf_x_freq = [];
mf_x_freq.low = trapmf(x_freq, [-1, 0, 0.05, 0.15]);
mf_x_freq.fine = trapmf(x_freq, [0.05, 0.15, 0.25, 0.35]);
mf_x_freq.high = trapmf(x_freq, [0.25, 0.40, 0.5, 1]);
[~, x_freq_ind] = min(abs(maj_f-x_freq));
x_freq_fuzzy = [mf_x_freq.low(x_freq_ind), mf_x_freq.fine(x_freq_ind),
    mf_x_freq.high(x_freq_ind)];
x_amp = 0:0.01:10;
mf_x_amp = [];
mf_x_amp.low = gbellmf(x_amp, [4, 10, -2]);
mf_x_amp.fine = gbellmf(x_amp, [5, 5, 8]);
[~, x_amp_ind] = min(abs(maj_P1-x_amp));
x_amp_fuzzy = [mf_x_amp.low(x_amp_ind), mf_x_amp.fine(x_amp_ind)];
% inference
y_likelihood_fuzzy = [0,0,0,0]; % very unlikely, unlikely, likely, very
     likely
y_likelihood_fuzzy(4) = max(y_likelihood_fuzzy(4), min(x_freq_fuzzy(2),
     x_amp_fuzzy(2)));
y_likelihood_fuzzy(3) = max(y_likelihood_fuzzy(3), min(x_freq_fuzzy(2),
     x_amp_fuzzy(1)));
y_likelihood_fuzzy(3) = max(y_likelihood_fuzzy(3), min(x_freq_fuzzy(3),
     x_amp_fuzzy(2)));
y_likelihood_fuzzy(3) = max(y_likelihood_fuzzy(3), min(x_freq_fuzzy(1),
     x_amp_fuzzy(2)));
y_likelihood_fuzzy(2) = max(y_likelihood_fuzzy(2), min(x_freq_fuzzy(3),
     x_amp_fuzzy(1)));
y_likelihood_fuzzy(1) = max(y_likelihood_fuzzy(2), min(x_freq_fuzzy(1),
```

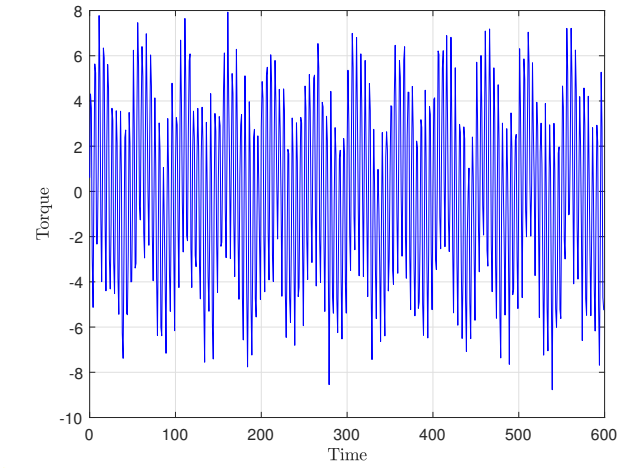


Figure 14.7.: Measured torque.

```
            x_amp_fuzzy(1)));
        % defizzification
        x_likelihood = 0:0.01:1;
        mf_x_likelihood = [];
        mf_x_likelihood.verynotlikely = gbellmf(x_likelihood, [0.2, 5, 0]);
        mf_x_likelihood.notlikely = gbellmf(x_likelihood, [0.15, 3, 0.3]);
        mf_x_likelihood.likely = gbellmf(x_likelihood, [0.15, 3, 0.6]);
        mf_x_likelihood.verylikely = gbellmf(x_likelihood, [0.3, 5, 1]);
        mf = max(min(y_likelihood_fuzzy(1), mf_x_likelihood.verynotlikely), ...
               max(min(y_likelihood_fuzzy(2), mf_x_likelihood.notlikely), ...
               max(min(y_likelihood_fuzzy(3), mf_x_likelihood.likely), ...
               min(y_likelihood_fuzzy(4), mf_x_likelihood.verylikely))));
        vibration_likelihood = defuzz(x_likelihood , mf, 'mom');
end
```

Running the above code gives the following results in Figs. 14.7 and 14.8. The function returns a likelihood of 0.84.

If increase the frequency from $0.2Hz$ to $0.45Hz$, i.e., let

```
measured_torque = 5*sin(2*pi*0.4*t);
measured_torque = measured_torque + 2*sin(2*pi*0.02*t);
measured_torque = measured_torque + normrnd(0, 1, [1, length(measured_torque)])
    ;
```

the likelihood becomes 0.60. If further drop the amplitude and let

```
measured_torque = 2*sin(2*pi*0.4*t);
measured_torque = measured_torque + 1*sin(2*pi*0.02*t);
measured_torque = measured_torque + normrnd(0, 1, [1, length(measured_torque)])
    ;
```

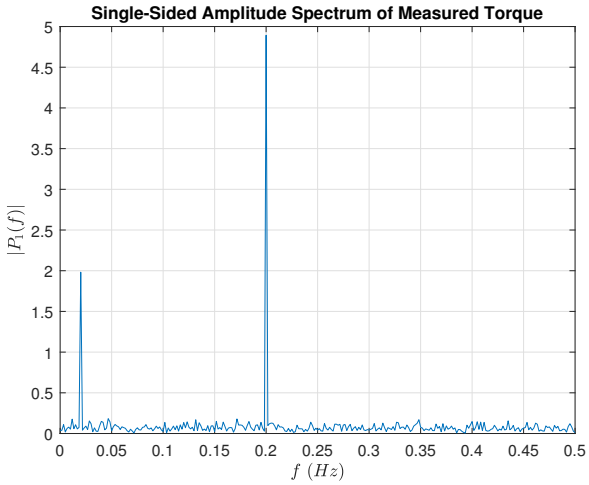


Figure 14.8.: Measured torque single-sided amplitude spectrum.

the likelihood further drops to 0.30. By an exhaustive search, we can plot likelihood as a function of oscillation frequency and amplitude as given in Fig. XXX.

Consider programming using MATLAB built-in tools to reproduce the above results. MATLAB provides variety choice of fuzzy inference systems. In this example, `mamfis` is used as follows.

```
vd_fis = mamfis( ...
       'NumInputs', 2, ...
       'NumInputMFs', [3, 2], ...
       'NumOutputs', 1, ...
       'NumOutputMFs', 4, ...
       'AddRule', 'none' ...
       );
% ios
vd_fis.Inputs(1).Name = 'torque_frequency';
vd_fis.Inputs(1).Range = [0, 0.5];
vd_fis.Inputs(1).MembershipFunctions(1).Name = 'low';
vd_fis.Inputs(1).MembershipFunctions(1).Type = 'trapmf';
vd_fis.Inputs(1).MembershipFunctions(1).Parameters = [-1, 0, 0.05, 0.15];
vd_fis.Inputs(1).MembershipFunctions(2).Name = 'fine';
vd_fis.Inputs(1).MembershipFunctions(2).Type = 'trapmf';
vd_fis.Inputs(1).MembershipFunctions(2).Parameters = [0.05, 0.15, 0.25, 0.35];
vd_fis.Inputs(1).MembershipFunctions(3).Name = 'high';
vd_fis.Inputs(1).MembershipFunctions(3).Type = 'trapmf';
vd_fis.Inputs(1).MembershipFunctions(3).Parameters = [0.25, 0.40, 0.5, 1];
vd_fis.Inputs(2).Name = 'torque_amplitude';
vd_fis.Inputs(2).Range = [0, 10];
vd_fis.Inputs(2).MembershipFunctions(1).Name = 'low';
```

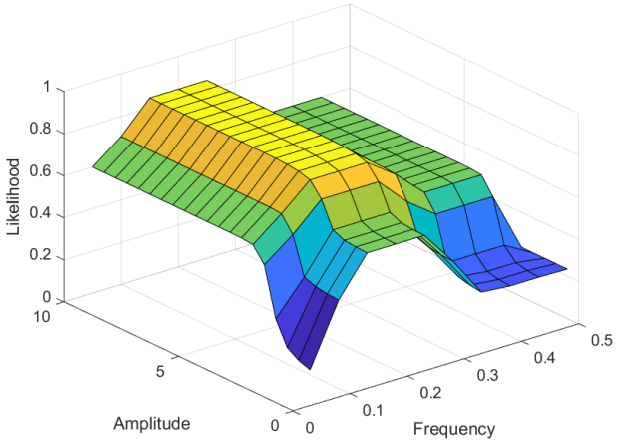


Figure 14.9.: Likelihood of vibration as a function of oscillation frequency and amplitude.

```
vd_fis.Inputs(2).MembershipFunctions(1).Type = 'gbellmf';
vd_fis.Inputs(2).MembershipFunctions(1).Parameters = [4, 10, -2];
vd_fis.Inputs(2).MembershipFunctions(2).Name = 'fine';
vd_fis.Inputs(2).MembershipFunctions(2).Type = 'gbellmf';
vd_fis.Inputs(2).MembershipFunctions(2).Parameters = [5, 5, 8];
plotmf(vd_fis, 'input', 1, 1000)
vd_fis.Outputs(1).Name = 'vibration_likelihood';
vd_fis.Outputs(1).Range = [0, 1];
vd_fis.Outputs(1).MembershipFunctions(1).Name = 'very_unlikely';
vd_fis.Outputs(1).MembershipFunctions(1).Type = 'gbellmf';
vd_fis.Outputs(1).MembershipFunctions(1).Parameters = [0.2, 5, 0];
vd_fis.Outputs(1).MembershipFunctions(2).Name = 'unlikely';
vd_fis.Outputs(1).MembershipFunctions(2).Type = 'gbellmf';
vd_fis.Outputs(1).MembershipFunctions(2).Parameters = [0.15, 3, 0.3];
vd_fis.Outputs(1).MembershipFunctions(3).Name = 'likely';
vd_fis.Outputs(1).MembershipFunctions(3).Type = 'gbellmf';
vd_fis.Outputs(1).MembershipFunctions(3).Parameters = [0.15, 3, 0.6];
vd_fis.Outputs(1).MembershipFunctions(4).Name = 'very_likely';
vd_fis.Outputs(1).MembershipFunctions(4).Type = 'gbellmf';
vd_fis.Outputs(1).MembershipFunctions(4).Parameters = [0.3, 5, 1];
plotmf(vd_fis, 'output', 1, 1000)
% rules
rules = ["if torque_frequency is fine and torque_amplitude is fine then
    vibration_likelihood is very_likely"; ...
       "if torque_frequency is fine and torque_amplitude is low then
           vibration_likelihood is likely"; ...
       "if torque_frequency is high and torque_amplitude is fine then
```

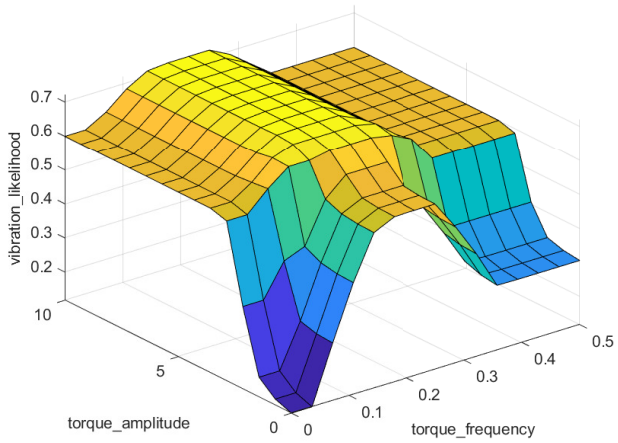


Figure 14.10.: Likelihood of vibration as a function of oscillation frequency and amplitude using `mamfis`.

```
            vibration_likelihood is likely"; ...
        "if torque_frequency is high and torque_amplitude is low then
            vibration_likelihood is unlikely"; ...
        "if torque_frequency is low and torque_amplitude is fine then
            vibration_likelihood is likely"; ...
        "if torque_frequency is low and torque_amplitude is low then
            vibration_likelihood is very_unlikely"; ...
        "if torque_amplitude is fine then vibration_likelihood is likely" ...
        ];
vd_fis = addRule(vd_fis, rules);
```

Use

```
evalfis(vd_fis, [<frequency>, <amplitude>])
```

to get the output of the fuzzy controller. Use

```
gensurf(vd_fis)
```

to obtain the plot of likelihood as a function of the oscillation frequency and amplitude as given in Fig. 14.10, which is similar with Fig. 14.9.

15. Multi-Agent Control System

This chapter discusses the problems that involve vector-valued design objectives. Multi-agent control system is often used to describe such problems, and game theory is used to solve them.

A. Matrix Algebra

This appendix chapter reviews matrix algebra, one of the most fundamental and widely used mathematical tools in this notebook. Notice that this chapter only serves as a brief summary, and cannot replace the linear algebra textbook or notebook.

A.1. Basic Operators

Let A, B be two square matrices of the same size. The following holds true

$$\begin{aligned} |A| &= |A^T| \\ |AB| &= |A||B| \end{aligned}$$

where $|\cdot|$ calculates the determinant of a square matrix. Furthermore, if both A, B are nonsingular,

$$(AB)^{-1} = B^{-1}A^{-1}$$

Let $A \in \mathbb{R}^{p\times n}$, $B \in \mathbb{R}^{n\times q}$. The following holds true

$$(AB)^T = B^T A^T$$

Furthermore, if $p = q$,

$$\text{trace}(AB) = \text{trace}(BA)$$

The *Kronecker product* of two matrix $A \in \mathbb{R}^{m\times n}$, $B \in \mathbb{R}^{p\times q}$ is defined as follows.

$$A \otimes B = [a_{ij}B] \in \mathbb{R}^{mp\times nq}$$

where a_{ij} is an element in A. Notice that some literature defines it as $[Ab_{ij}]$ instead, which is different from the earlier definition.

The stacking operator is defined as follows. Let $A \in \mathbb{R}^{m\times n}$,

$$s(A) = \begin{bmatrix} a_{1,1} \\ \vdots \\ a_{m,1} \\ \vdots \\ a_{1,n} \\ \vdots \\ a_{m,n} \end{bmatrix}$$

i.e., it stacks the columns of A into a column vector.

The following holds true

$$s(ABC) = s(C^T \otimes A)s(B)$$

Let $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{p \times q}$,

$$|A \otimes B| = |A|^p |B|^m$$

If λ_i is an eigenvalue of A with eigenvector v_i, then λ_i^{-1} is an eigenvalue of A^{-1} with the same eigenvector. This is because

$$Av_i = \lambda_i v_i$$

implies that

$$\lambda^{-1} v_i = A^{-1} v_i$$

For another matrix B with eigenvalue μ_i with respect to eigenvector w_i, $\lambda_i \mu_i$ is an eigenvalue of $A \otimes B$ w.r.t eigenvector $v_i \otimes w_i$.

The well-known matrix inversion lemma is

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U\left(C^{-1} + VA^{-1}U\right)^{-1} VA^{-1}$$

whose proof can be obtained using blockwise matrix inversion, which is neglected here.

A.2. Partitioned Matrix

The following matrix A is *block diagonal* if

$$A = \begin{bmatrix} \ddots & & 0 \\ & A_{ii} & \\ 0 & & \ddots \end{bmatrix}$$

In this case, A can be denoted by $A = \mathrm{diag}(\ldots, A_{ii}, \ldots)$. Furthermore, if A_{ii} is square matrix, A becomes square matrix, and

$$|A| = \Pi_i |A_{ii}|$$

If A is nonsingular,

$$A^{-1} = \mathrm{diag}(\ldots, A_{ii}^{-1}, \ldots)$$

A.3. Quadratic Forms

Let $x \in \mathbb{R}^n$ a (column) vector. The square of the Euclidean norm is

$$||x||^2 = x^T x$$

Let S be a nonsingular transformation, and $P = S^T S$ a symmetric matrix,

$$\begin{aligned} ||x||_P^2 &= ||Sx||^2 \\ &= x^T S^T S x \\ &= x P x^T \end{aligned}$$

The quadratic form of x looks like the following

$$x^T Q x \tag{A.1}$$

where Q is real. Notice that Q can be divided into symmetric and asymmetric parts

$$Q = Q_s + Q_a$$

where

$$\begin{aligned} Q_s &= (Q + Q^T)/2 \\ Q_a &= (Q - Q^T)/2 \end{aligned} \tag{A.2}$$

and (A.1) becomes

$$\begin{aligned} x^T Q x &= x^T (Q_s + Q_a) x \\ &= x^T Q_s x \end{aligned}$$

where notice that $x^T Q_a x = 0$. This is because $x^T Q_a x = x^T Q_a^T x = -x^T Q_a x$ since $x^T Q_a x$ is a scalar and from (A.2) $Q_a^T = -Q_a$.

In (A.1), we say $Q > 0$ is positive definite if for all nonzero x, $x^T Q x > 0$. This is equivalent of saying that for all eigenvalues λ_i of Q, $\lambda_i > 0$. Semi-positive definite, semi-negative definite, and negative definite can be defined similarly.

A.4. Matrix Calculus

Matrix calculus defines how the differential of a vector/matrix w.r.t another vector/matrix is defined.

Consider s being a scalar and $x \in \mathbb{R}^n$ a factor. Let x be a vector function of s. The differential of x w.r.t. s is defined as follows.

$$\frac{dx}{ds} = \begin{bmatrix} \frac{dx_1}{ds} \\ \vdots \\ \frac{dx_n}{ds} \end{bmatrix}$$

If s is a function of x, the gradient of s w.r.t. x has two forms, the gradient form where the result is a column vector, and the Jacobian form where the result is a row vector. They are transpose of each other. For example, the gradient form is

$$\frac{\partial s}{\partial x} = \nabla s = \begin{bmatrix} \frac{\partial s}{\partial x_1} \\ \vdots \\ \frac{\partial s}{\partial x_n} \end{bmatrix} \tag{A.3}$$

and

$$ds = (\nabla s)^T dx$$

where $dx = [dx_1, \ldots, dx_n]^T$. The Jacobian form denoted by $J(s)$ is defined as the transpose of (A.3). Furthermore, if s is a function of multiple vectors, for example both x and y,

$$ds = \left(\frac{\partial s}{\partial x}\right)^T dx + \left(\frac{\partial s}{\partial y}\right)^T dy$$

The *Hessian* of a scalar s w.r.t. a vector x is the second derivative

$$\frac{\partial^2 s}{\partial x^2} = \left[\frac{\partial^2 s}{\partial x_i \partial x_j}\right]$$

Let $f \in \mathbb{R}^m$ be a vector function of vector $x \in \mathbb{R}^n$. The jacobian of f w.r.t. x is a $m \times n$ matrix given by

$$\frac{\partial f}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Let A be a non-singular square matrix of function of time t.

$$\frac{d}{dt}\left(A^{-1}\right) = -A^{-1}\frac{d}{dt}(A)\,A^{-1}$$

Notice that in references, the differential of a variable x or a matrix A w.r.t. time can also be denoted as $\dot{x}$ and $\dot{A}$, respectively.

Let y, A be constant vector and matrix, respectively, with dimensions so that the

following expressions make sense. Then we have

$$\begin{aligned}
&\frac{\partial}{\partial x}\left(y^T x\right) = \frac{\partial}{\partial x}\left(x^T y\right) = y \\
&\frac{\partial}{\partial x}\left(Ax\right) = A \\
&\frac{\partial}{\partial x}\left(y^T Ax\right) = \frac{\partial}{\partial x}\left(x^T A^T y\right) = A^T y \\
&\frac{\partial}{\partial x}\left(y^T f(x)\right) = \frac{\partial}{\partial x}\left(f(x)^T y\right) = \left(\frac{\partial f}{\partial x}\right)^T y \\
&\frac{\partial}{\partial x}\left(x^T Ax\right) = Ax + A^T x \\
&\frac{\partial^2}{\partial x^2}\left(x^T Ax\right) = A + A^T
\end{aligned}$$

In a special case where Q is symmetric,

$$\begin{aligned}
\frac{\partial}{\partial x}\left(x^T Qx\right) &= 2Qx \\
\frac{\partial^2}{\partial x^2}\left(x^T Qx\right) &= 2Q \\
\frac{\partial}{\partial x}\left((x-y)^T Q(x-y)\right) &= 2Q(x-y) \\
\frac{\partial^2}{\partial x^2}\left((x-y)^T Q(x-y)\right) &= 2Q
\end{aligned}$$

Furthermore, if f, g are two vector functions of vector x,

$$\frac{\partial}{\partial x}\left(f^T g\right) = \left(\frac{\partial}{\partial x} f\right)^T g + \left(\frac{\partial}{\partial x} g\right)^T f$$

B. Brief Introduction to Calculus of Variations

Calculus of variations studies the change of the value of a functional w.r.t the choice of function. A commonly seen objective of calculus of variations is to find the optimal function among a set of functions (usually defined by restrictions) that maximizes or minimizes a functional.

Notice that under the scope of this notebook, only a few ideas from calculus of variations are needed. Therefore, only a brief introduction is introduced. More details about calculus of variations can be found in another notebook *A Notebook on Calculus*.

B.1. Problem Formulation

Consider a cost function defined by

$$J = \int_0^T h\left(x(t), t\right) dt \tag{B.1}$$

where $x(t)$ is a function of t. The objective is to find $x(t)$ in the searching space that minimizes the above cost function.

B.2. Brief Discussion of the Solution

Similar with static optimization, the solution to (B.1) can be obtained by studying how the change of each element in the cost function would affect the change of the value of the cost function, i.e., how $dx(t)$ and dt affect dJ.

There are some obvious differences of (B.1) comparing with discrete time optimization such as the following

$$J = \sum_{k=0}^{N-1} h\left(x(k), u(k)\right) \tag{B.2}$$

The major differences are

1. In (B.1) integration is used, while in (B.2) sum is used.
2. In (B.1), the two variables $x(t)$ and t are correlated, while in (B.2), $x(t)$ and $u(t)$ can be treated as independent variables, with the system dynamics $x(k+1) = f\left(x(k) + u(k)\right)$ the equality constraint.

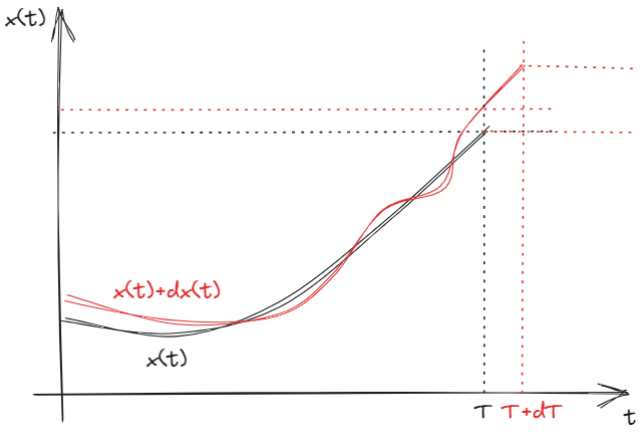


Figure B.1.: A demonstration of the calculation of $dx(T)$ from calculus of variations perspective.

Integration and sum are in the essence describing the same thing, where integration a limiting case of sum. Therefore, this first difference does not affect much how we would proceed with the calculation. It is the second difference that forms the main challenge.

Consider $x(t)$ being a function of t defined from $t = 0$ to $t = T$. Consider a neighborhood function $x(t) + dx(t)$. Notice that the neighborhood function $x(t) + dx(t)$ is not necessarily cut-off at $t = T$. Without loosing generality, assume that the neighborhood function is stretched from $t = T$ to $t = T + dT$. The final value of the original and the neighborhood functions can be bridged as follows.

$$[x(t) + dx(t)]|_{t=T+dT} - x(T) \quad = \quad \delta x(T) + \dot{x}(T)dT$$

where $\delta x(T)$ describes a portion of the final value increment of the neighborhood function irrelevant to change in time, and $\dot{x}(T)dT$ the portion caused due to the change in time. Notice that since $x(t)$ and $x(t) + dx(t)$ are close enough, their slope at $t = T$ shall be approximately the same. This is demonstrated by Fig. B.1.

Another relation that describes dJ in (B.1) is given below. This is known as the Leibniz's rule for functionals.

$$dJ \quad = \quad h\left(x(T), T\right) dT - h\left(x(0), 0\right) dt_0 + \int_{t=0}^{T} \left(h_x^T\left(x(t), t\right) \delta x(t)\right) dt$$

where dT and dt_0 describe the stretch of the neighborhood function at the end and the beginning w.r.t. t, respectively, and

$$h_x \quad = \quad \frac{\partial h}{\partial x}$$

Bibliography

[1] Karl J Åström and Björn Wittenmark. *Adaptive control*. Courier Corporation, 2013.

[2] Masaharu Mizumoto and Kokichi Tanaka. Fuzzy sets and their operations. *Information and Control*, 48(1):30–48, 1981.